AUCA · Big Data Analytics

Mobile Money Investigations with PostgreSQL and Neo4j

Advanced Big Data Analytics · Assignment 2

Group 2 · Lecturer: Temitope Oguntande

Member	Student ID
Murenzi Prince	101368
Mashami Pacifique	101406
Delice Sebaganwa Shami	101371

Code repository: **https://github.com/murenziprince/mobile-money-investigations**
Matching ZIP: **mobile_money_project.zip**

Summary of findings

- **Correctness.** 61 of 61 cases (Q1–Q7) give identical normalised answers in PostgreSQL, Neo4j and an independent CSV-based reference; 8/8 Q7 supporting paths are valid in each engine.
- **Reporting.** PostgreSQL answers Q1–Q3 in 1–13,000 ms; Neo4j needs 15–50 ms for simple lookups and times out on the 20-million-row Q2 aggregation (5/5 runs over 60 s).
- **Traversal.** The baseline SQL for Q7 was 741× slower than Neo4j for B at bound 6 (40.7 s vs 54.9 ms). A bidirectional rewrite brought PostgreSQL to 94.9 ms, erasing most of the graph advantage; this was an algorithm effect, not a modelling effect.
- **Concurrency.** Neo4j gives 2× (Q4 depth 10) to 4× (Q7, one client) the throughput of PostgreSQL, but its Q7 throughput halves and its p95 latency exceeds PostgreSQL's at 16 clients on this 4-thread laptop.
- **Investigation.** Only accounts A, B and C have a rapid-onward chain; A and C also have a circular sequence. Device sharing and network size do not separate the pairs, so A, B and C come before D, E and F. No conclusion of fraud is drawn.

Contents

1 Capture the data · 2 Build PostgreSQL · 3 Build Neo4j · 4 Routine business questions (Q1–Q3) · 5 Connected accounts (Q4–Q7) · 6 Correctness and performance · 7 Concurrent requests · 8 Decisions and individual notes · 9 Reproducibility.

All screenshots show the command, the result, the taskbar and the clock. Full outputs, raw repetitions and plans are in the code submission (`results/`, `plans/`).

1 Capture the data

1.1 Workspace

Item	Recorded value (results/environment.json)
Operating system	Windows 11 (10.0.22631)
CPU	Intel Core i7-7600U, 2 cores / 4 logical (Intel64 Family 6 Model 142)
RAM	8.46 GB (Task Manager: 7.9 GB usable)
Python / drivers	Python 3.14.5; psycopg 3.3.6; neo4j driver 6.4.1; numpy 2.5.4; matplotlib 3.11.2
PostgreSQL	18.6 (x86_64-windows, MSVC 19.44), database `mobile_money`; TimeZone **Africa/Kigali**
PostgreSQL memory	shared_buffers 128MB, work_mem 4MB, effective_cache_size 4GB, maintenance_work_mem 64MB, max_parallel_workers_per_gather 2, max_connections 100 (installer defaults)
Neo4j	Community 5.26.32 (Kernel 5.26.32), default database `neo4j`, Java 17 (Temurin)
Neo4j memory	heap initial/max 3.52 GiB, page cache 1.95 GiB, transaction memory cap 2.46 GiB (dbms.memory.transaction.total.max)

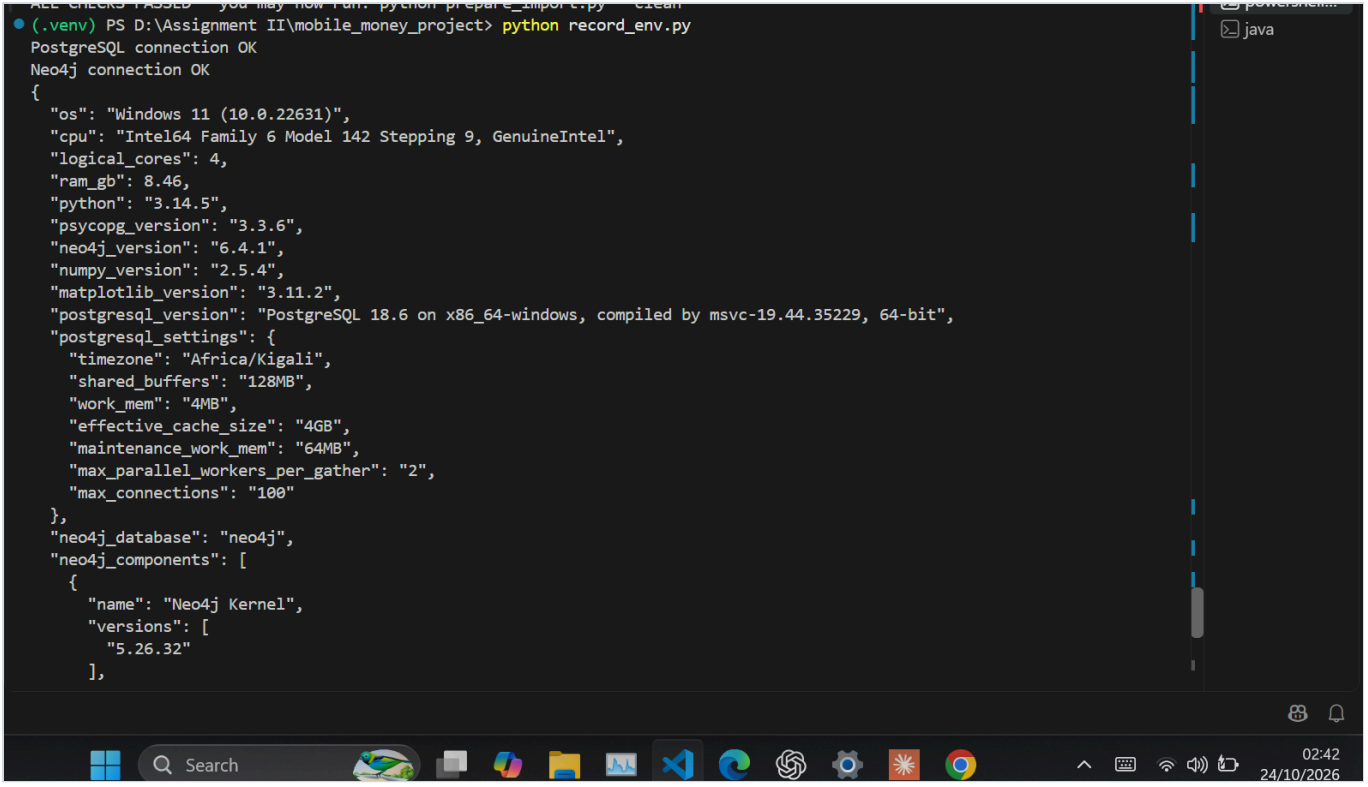


Fig 1.1 — `record_env.py`: both connections OK, OS/CPU/RAM, versions, PostgreSQL settings (the Neo4j part is in environment.json).

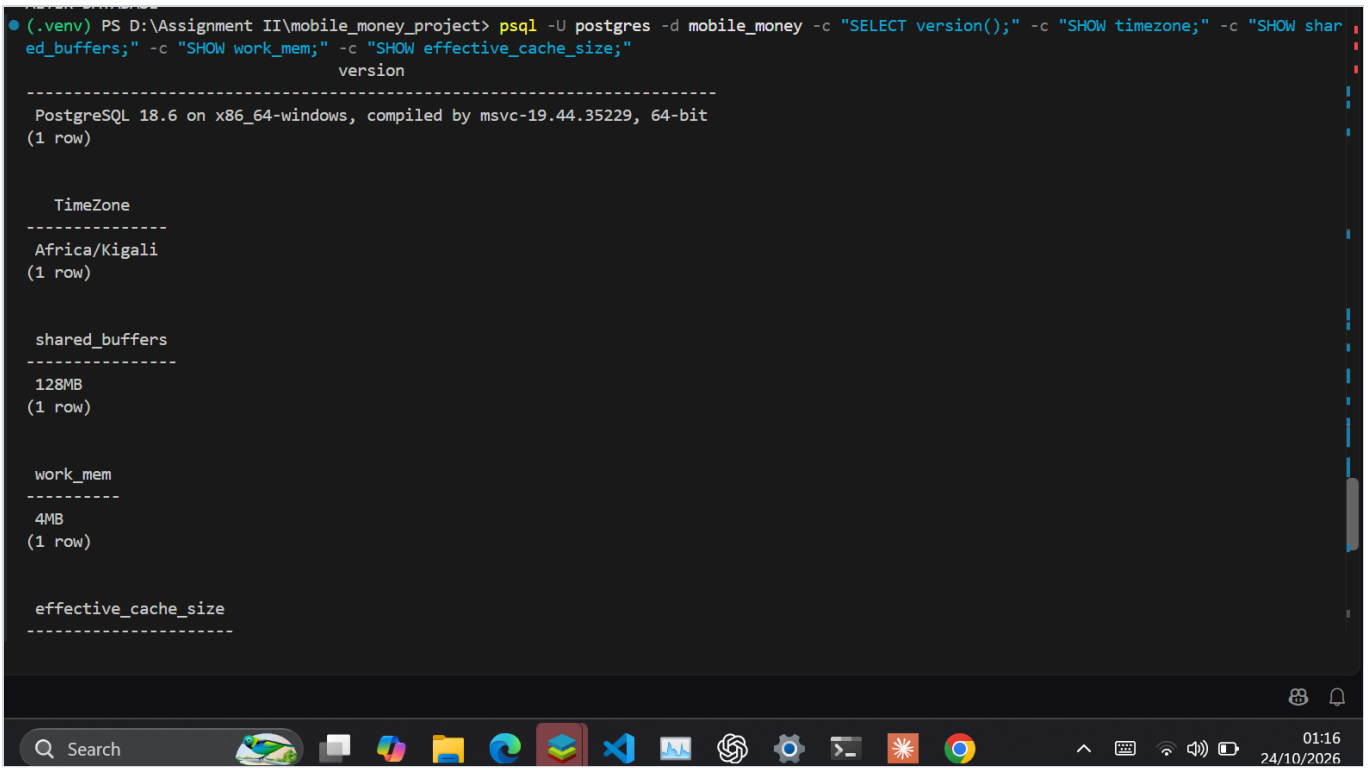


Fig 1.2 — psql: PostgreSQL 18.6, TimeZone Africa/Kigali, shared_buffers 128MB, work_mem 4MB.

1.2 capture.py

The generator is imported unchanged (SHA-256 verified, Fig 9.1). Each stream is written progressively with `csv.DictWriter`; rows are counted as consumed and written, transfers are logged every million, and the audit accumulates count, min/max `occurred_at` and the sum of `amount_rwf`.

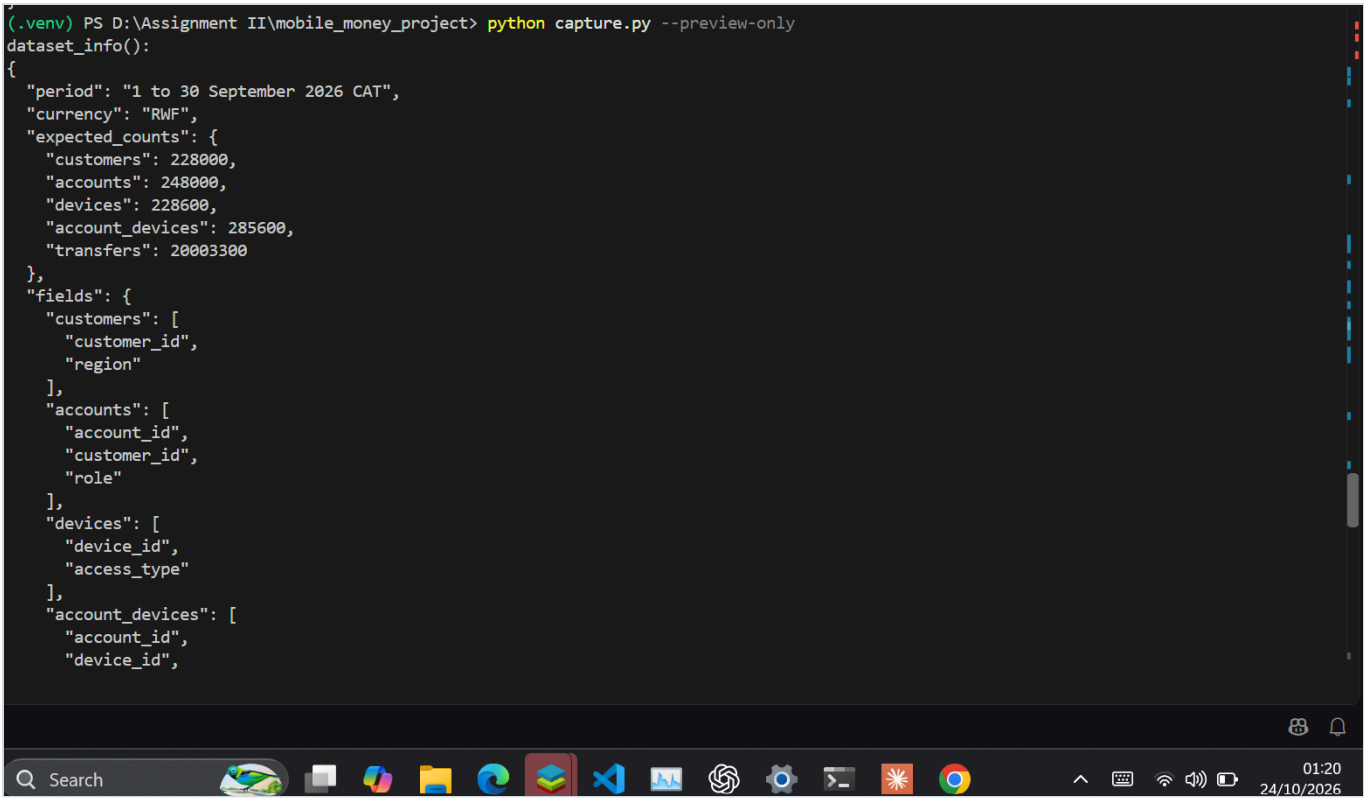


Fig 1.3 — `capture.py --preview-only`: `dataset_info()` and `FIELDS`.

```
(.venv) PS D:\Assignment II\mobile_money_project> python capture.py --preview-only
{'device_id': 6, 'access_type': 'handset'}
{'device_id': 7, 'access_type': 'handset'}
{'device_id': 8, 'access_type': 'handset'}
{'device_id': 9, 'access_type': 'handset'}
{'device_id': 10, 'access_type': 'handset'}

--- first 10 records of account_devices ---
{'account_id': 1, 'device_id': 1, 'first_day': 0, 'last_day': 29, 'observations': 11}
{'account_id': 2, 'device_id': 1, 'first_day': 0, 'last_day': 29, 'observations': 12}
{'account_id': 3, 'device_id': 1, 'first_day': 0, 'last_day': 29, 'observations': 13}
{'account_id': 4, 'device_id': 1, 'first_day': 0, 'last_day': 29, 'observations': 14}
{'account_id': 5, 'device_id': 1, 'first_day': 0, 'last_day': 29, 'observations': 15}
{'account_id': 6, 'device_id': 6, 'first_day': 0, 'last_day': 29, 'observations': 16}
{'account_id': 7, 'device_id': 7, 'first_day': 0, 'last_day': 29, 'observations': 17}
{'account_id': 8, 'device_id': 8, 'first_day': 11, 'last_day': 29, 'observations': 18}
{'account_id': 8, 'device_id': 221105, 'first_day': 0, 'last_day': 10, 'observations': 10}
{'account_id': 9, 'device_id': 9, 'first_day': 0, 'last_day': 29, 'observations': 19}

--- first 10 records of transfers ---
{'transfer_id': 1, 'sender_id': 188503, 'receiver_id': 187907, 'occurred_at': 1788235200, 'amount_rwf': 6700}
{'transfer_id': 2, 'sender_id': 151724, 'receiver_id': 152176, 'occurred_at': 1788235200, 'amount_rwf': 6700}
{'transfer_id': 3, 'sender_id': 199087, 'receiver_id': 199200, 'occurred_at': 1788235200, 'amount_rwf': 6800}
{'transfer_id': 4, 'sender_id': 139778, 'receiver_id': 140682, 'occurred_at': 1788235200, 'amount_rwf': 6300}
{'transfer_id': 5, 'sender_id': 189061, 'receiver_id': 189287, 'occurred_at': 1788235200, 'amount_rwf': 2600}
{'transfer_id': 6, 'sender_id': 85014, 'receiver_id': 246711, 'occurred_at': 1788235200, 'amount_rwf': 30600}
{'transfer_id': 7, 'sender_id': 225854, 'receiver_id': 225258, 'occurred_at': 1788235200, 'amount_rwf': 3800}
{'transfer_id': 8, 'sender_id': 69746, 'receiver_id': 240010, 'occurred_at': 1788235200, 'amount_rwf': 16700}
{'transfer_id': 9, 'sender_id': 7642, 'receiver_id': 240005, 'occurred_at': 1788235200, 'amount_rwf': 900}
{'transfer_id': 10, 'sender_id': 32477, 'receiver_id': 246273, 'occurred_at': 1788235200, 'amount_rwf': 3900}
```

Search 01:21 24/10/2026

Fig 1.4 — ten preview records per stream (devices, account_devices and transfers visible; customers and accounts are in the full output).

```
(.venv) PS D:\Assignment II\mobile_money_project> python capture.py
  transfers: 4,000,000 rows, 72.6s elapsed
  transfers: 5,000,000 rows, 107.8s elapsed
  transfers: 6,000,000 rows, 122.0s elapsed
  transfers: 7,000,000 rows, 136.3s elapsed
  transfers: 8,000,000 rows, 151.5s elapsed
  transfers: 9,000,000 rows, 164.7s elapsed
  transfers: 10,000,000 rows, 178.2s elapsed
  transfers: 11,000,000 rows, 191.4s elapsed
  transfers: 12,000,000 rows, 204.9s elapsed
  transfers: 13,000,000 rows, 218.0s elapsed
  transfers: 14,000,000 rows, 231.3s elapsed
  transfers: 15,000,000 rows, 244.8s elapsed
  transfers: 16,000,000 rows, 258.2s elapsed
  transfers: 17,000,000 rows, 271.1s elapsed
  transfers: 18,000,000 rows, 284.8s elapsed
  transfers: 19,000,000 rows, 297.8s elapsed
  transfers: 20,000,000 rows, 310.7s elapsed
  transfers: consumed=20,003,300 written=20,003,300 expected=20,003,300 OK (310.78s, 0.7802 GB)

=== Five-stream totals ===
customers        rows=     228,000    0.0036 GB       2.15s
accounts         rows=     248,000    0.0052 GB       2.41s
devices          rows=     228,600    0.0036 GB       2.35s
account_devices  rows=     285,600    0.0063 GB       4.31s
transfers        rows=  20,003,300    0.7802 GB     310.78s
transfer audit: count=20,003,300 min_occurred_at=1788235200 max_occurred_at=1790798399 sum_amount_rwf=231,482,327,870
total source records: 20,993,500 (handout: 20,993,500)
total CSV size: 0.7988 GB | total capture seconds: 322.01
(.venv) PS D:\Assignment II\mobile_money_project>
```

Search 01:28 24/10/2026

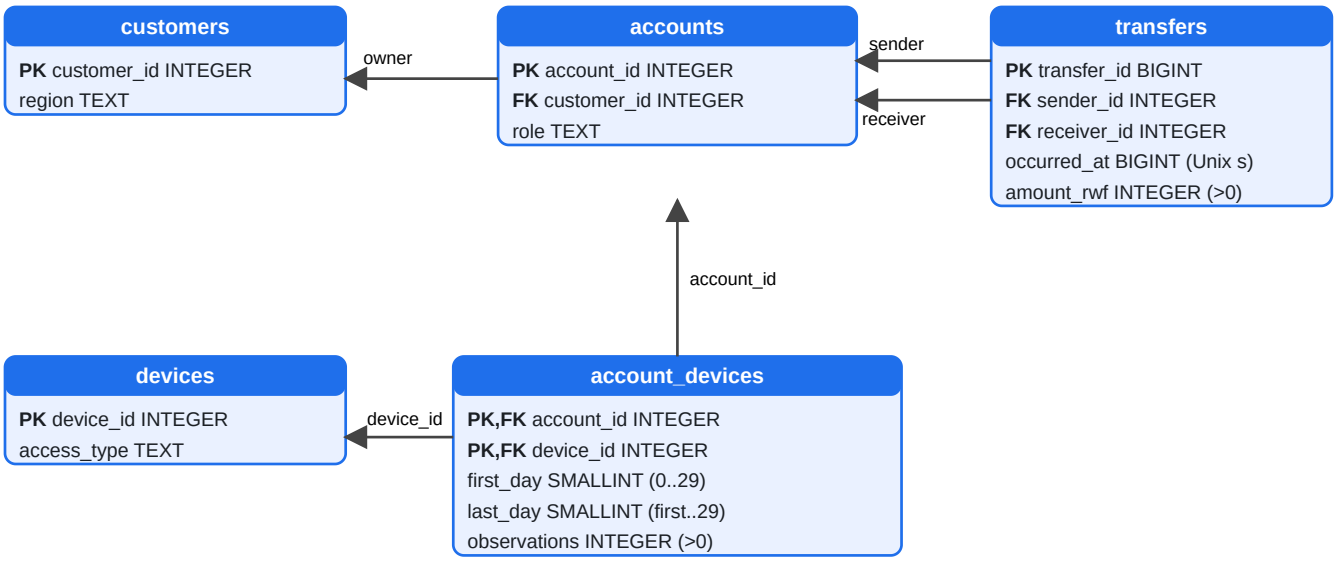
Fig 1.5 — full capture: five streams match the handout counts; 20,993,500 records; 0.7988 GB; 322.01 s.

Stream	Rows	Size (GB)	Seconds
customers	228,000	0.0036	2.15
accounts	248,000	0.0052	2.41
devices	228,600	0.0036	2.35
account_devices	285,600	0.0063	4.31
transfers	20,003,300	0.7802	310.78
Total	**20,993,500**	**0.7988**	**322.01**

Transfer audit: count 20,003,300; min occurred_at 1,788,235,200; max 1,790,798,399; sum amount 231,482,327,870 RWF.

2 Build PostgreSQL

2.1 Schema



Arrows point from the referencing (FK) column to the referenced primary key. All columns are NOT NULL.

Fig 2.1 — PostgreSQL schema (primary keys, foreign keys and checks).

Table.column	Meaning and unit
customers.customer_id / region	Customer identifier; home region (Kigali, Northern, Southern, Eastern, Western)
accounts.account_id / customer_id / role	Wallet identifier; owning customer; role person, merchant or agent
devices.device_id / access_type	Device identifier; handset or public terminal
account_devices.first_day, last_day	Inclusive day numbers 0–29 counted from 1 Sep 2026 (CAT); interval in which the device was observed for the account
account_devices.observations	Count of observations in that interval (>0); does not say which device sent a payment
transfers.transfer_id	Unique payment identifier (BIGINT)
transfers.sender_id / receiver_id	Paying and receiving account
transfers.occurred_at	Unix seconds (UTC epoch); shown in CAT = UTC+2
transfers.amount_rwf	Whole Rwandan francs, always >0 (CHECK); not a balance

```
(.venv) PS D:\Assignment II\mobile_money_project> psql -U postgres -d mobile_money -f sql/04_show_schema.sql
=== table definitions ===
                Table "public.customers"
    Column     |  Type   | Collation | Nullable | Default
---------------+---------+-----------+----------+---------
 customer_id   | integer |           | not null |
 region        | text    |           | not null |
Indexes:
    "customers_pkey" PRIMARY KEY, btree (customer_id)
Referenced by:
    TABLE "accounts" CONSTRAINT "accounts_customer_fk" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)


                Table "public.devices"
    Column     |  Type   | Collation | Nullable | Default
---------------+---------+-----------+----------+---------
 device_id     | integer |           | not null |
 access_type   | text    |           | not null |
Indexes:
    "devices_pkey" PRIMARY KEY, btree (device_id)
Referenced by:
    TABLE "account_devices" CONSTRAINT "ad_device_fk" FOREIGN KEY (device_id) REFERENCES devices(device_id)


                Table "public.accounts"
    Column     |  Type   | Collation | Nullable | Default
---------------+---------+-----------+----------+---------
 account_id    | integer |           | not null |
```

Fig 2.2 — `psql -f sql/04_show_schema.sql`: table definitions with column types and NOT NULL (customers and devices shown; all five tables are in the full output in the code submission).

```
(.venv) PS D:\Assignment II\mobile_money_project> psql -U postgres -d mobile_money -f sql/04_show_schema.sql

=== constraints (convalidated = t means validated) ===
   table_name    |             conname              | contype | convalidated |                                                  definition
-----------------+----------------------------------+---------+--------------+---------------------------------------------------------------------------------------------------
 account_devices | account_devices_days_ck          | c       | t            | CHECK (((0 <= first_day) AND (first_day <= last_day) AND (last_day <= 29)))
 account_devices | account_devices_observations_check | c       | t            | CHECK ((observations > 0))
 account_devices | ad_account_fk                    | f       | t            | FOREIGN KEY (account_id) REFERENCES accounts(account_id)
 account_devices | ad_device_fk                     | f       | t            | FOREIGN KEY (device_id) REFERENCES devices(device_id)
 account_devices | account_devices_pkey             | p       | t            | PRIMARY KEY (account_id, device_id)
 accounts        | accounts_customer_fk             | f       | t            | FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
 accounts        | accounts_pkey                    | p       | t            | PRIMARY KEY (account_id)
 customers       | customers_pkey                   | p       | t            | PRIMARY KEY (customer_id)
 devices         | devices_pkey                     | p       | t            | PRIMARY KEY (device_id)
 transfers       | transfers_amount_rwf_check       | c       | t            | CHECK ((amount_rwf > 0))
 transfers       | transfers_receiver_fk            | f       | t            | FOREIGN KEY (receiver_id) REFERENCES accounts(account_id)
 transfers       | transfers_sender_fk              | f       | t            | FOREIGN KEY (sender_id) REFERENCES accounts(account_id)
 transfers       | transfers_pkey                   | p       | t            | PRIMARY KEY (transfer_id)
(13 rows)


=== indexes ===
    tablename    |          indexname          |                                                
```

2.2 Bulk load and checks

Step	Seconds
Bulk load with \copy (5 tables, in dependency order)	109.69
Foreign keys + 5 indexes (incl. BRIN) + ANALYZE	170.83

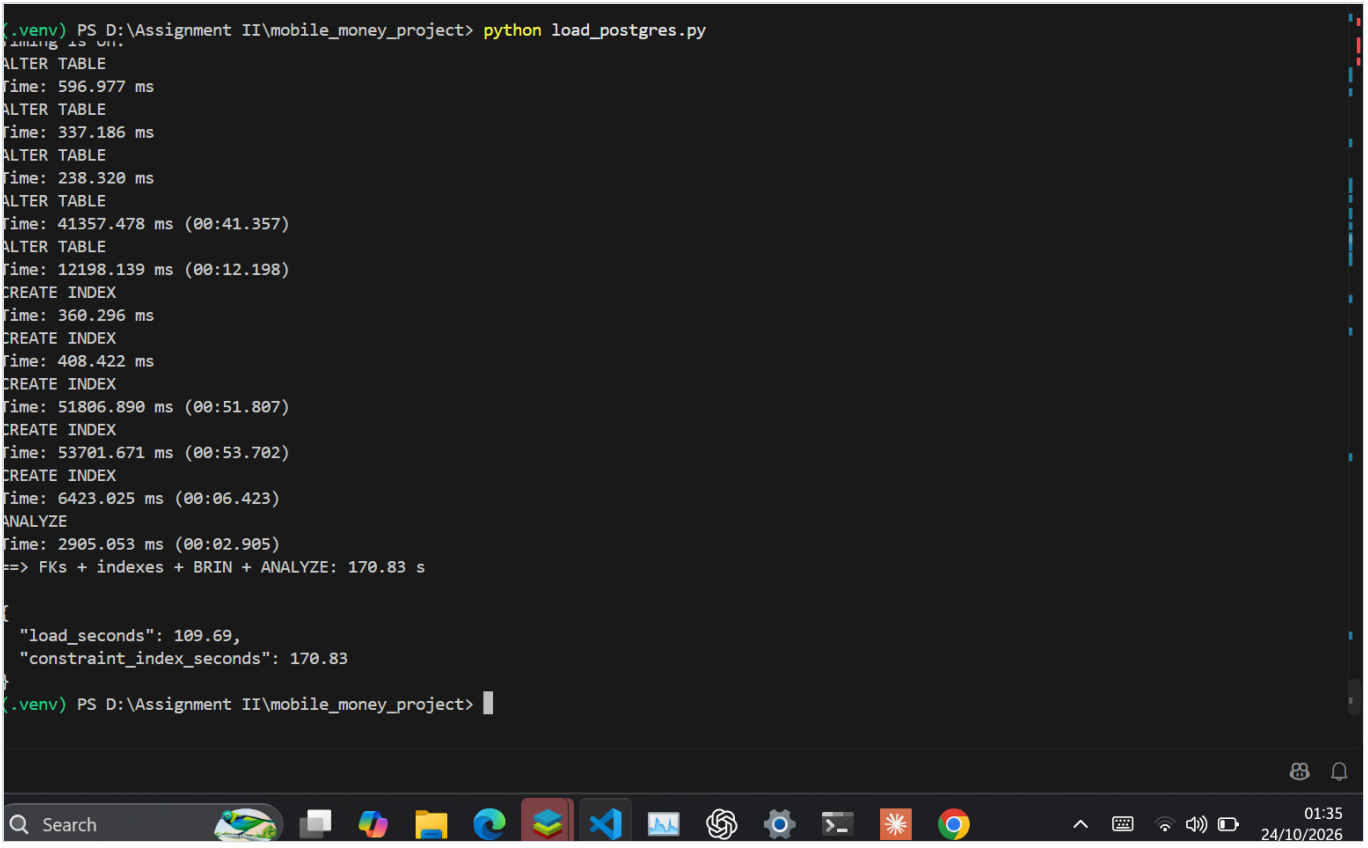


Fig 2.4 — `load_postgres.py`: end of the run showing index timings and the two totals, kept separate.

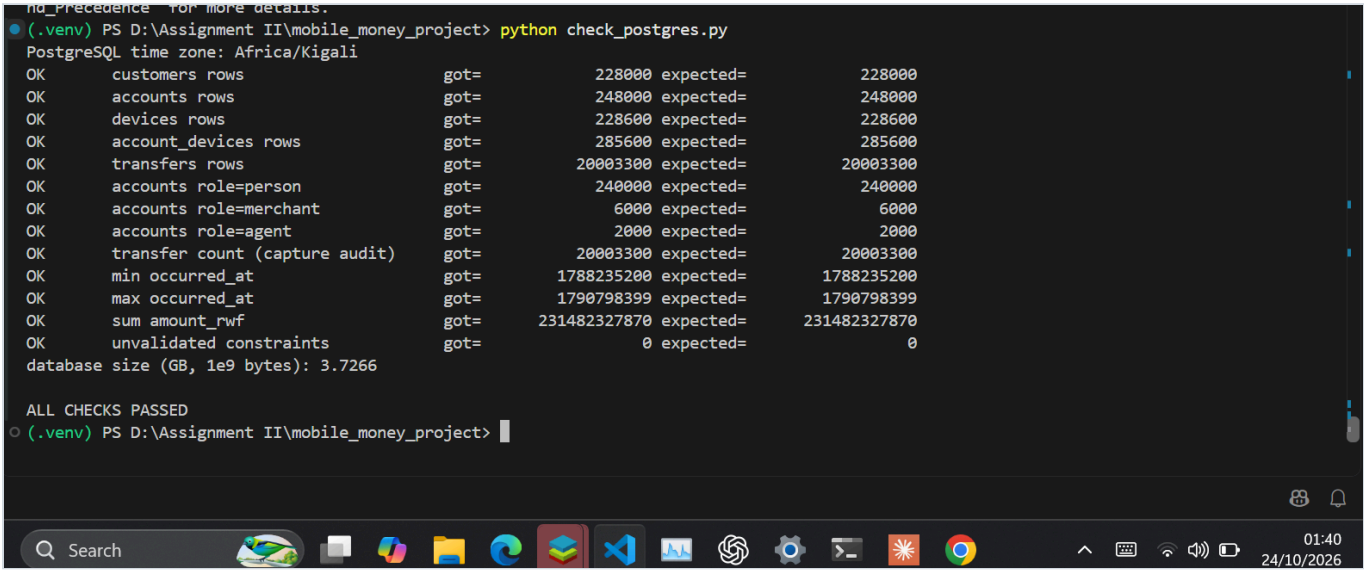


Fig 2.5 — `check_postgres.py`: all row counts, 240,000/6,000/2,000 roles, audit totals, 0 unvalidated constraints, database 3.7266 GB.

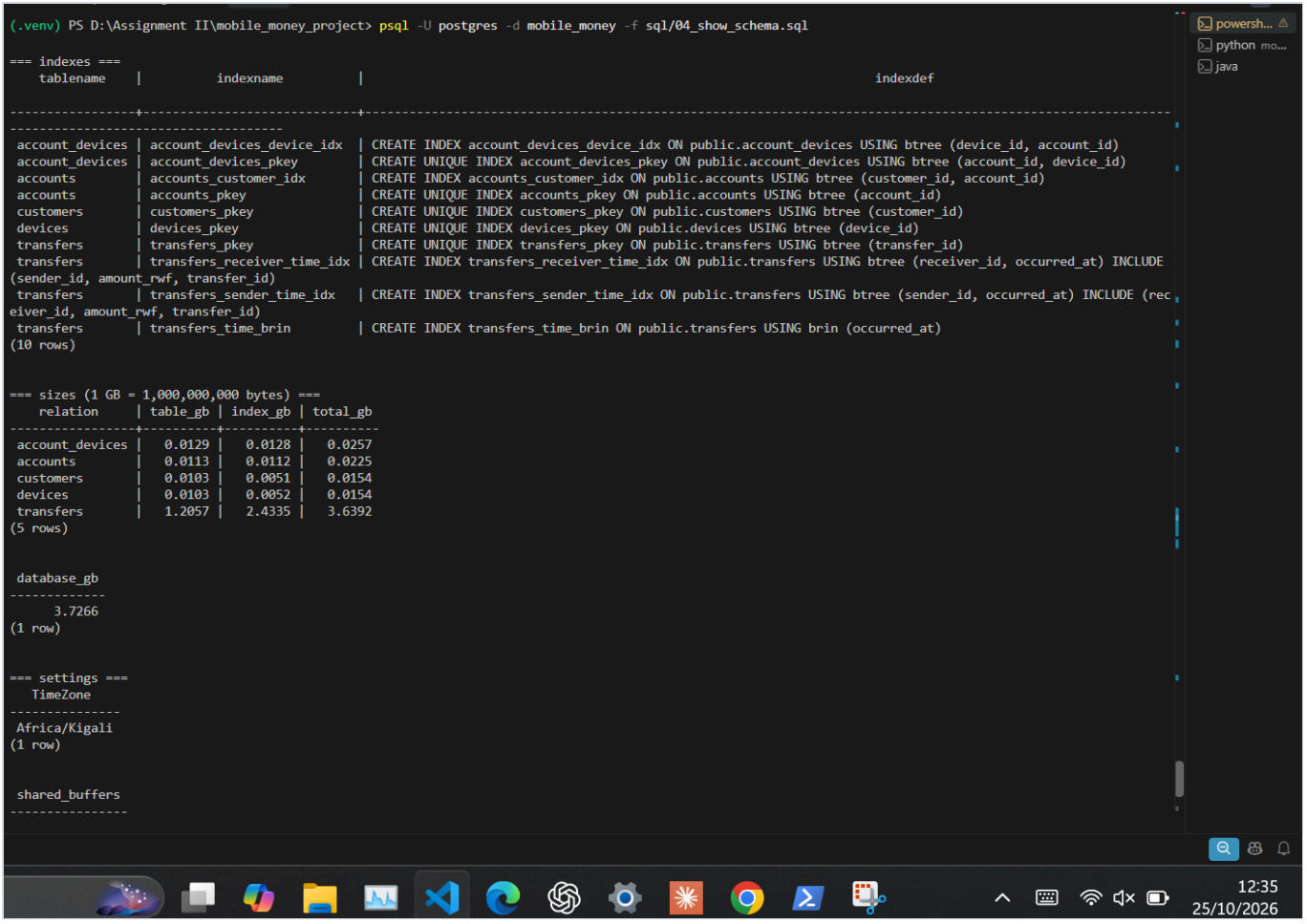


Fig 2.6 — the required indexes (accounts_customer_idx, account_devices_device_idx, the two transfers INCLUDE indexes and the BRIN index), data and index sizes in GB (1 GB = 1,000,000,000 bytes), database 3.7266 GB, TimeZone Africa/Kigali.

For transfers the table is 1.2057 GB and its indexes 2.4335 GB (total 3.6392 GB): the two covering indexes are larger than the data, which is the price of fast per-account lookups.

3 Build Neo4j

3.1 Graph model

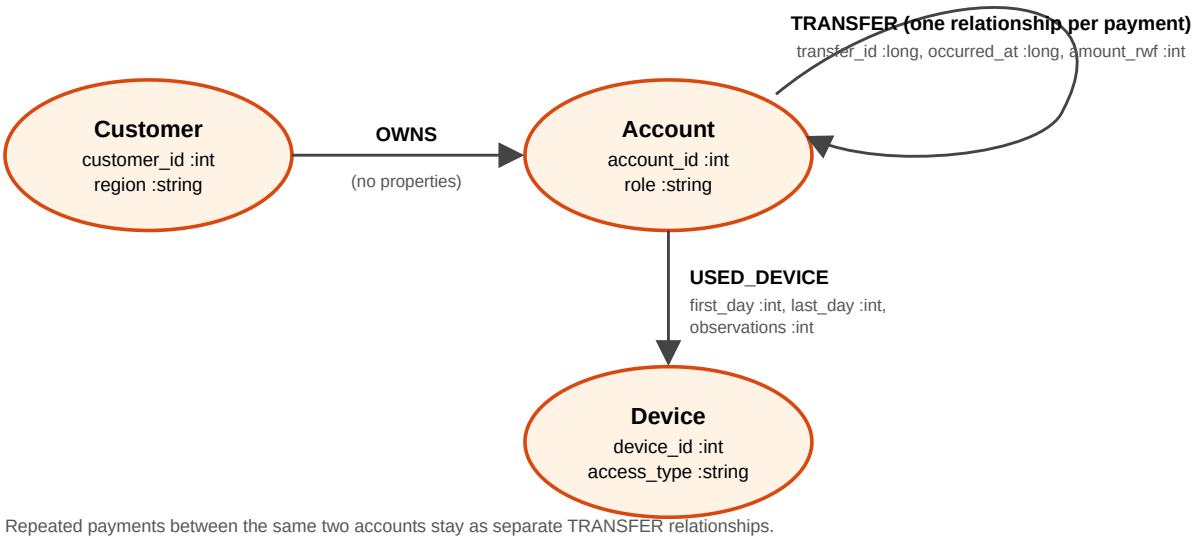


Fig 3.1 — Graph schema.

Source field	Graph element
customers.customer_id, region	(:Customer {customer_id:int, region:string})
accounts.account_id, role	(:Account {account_id:int, role:string})
accounts.customer_id	(:Customer)-[:OWNS]→(:Account) — derived file owns.csv (customer_id, account_id order); the column is ignored in the Account node file
devices.device_id, access_type	(:Device {device_id:int, access_type:string})
account_devices.account_id, device_id	(:Account)-[:USED_DEVICE]→(:Device)
account_devices.first_day, last_day, observations	properties on USED_DEVICE (int)
transfers.sender_id, receiver_id	start and end node of (:Account)-[:TRANSFER]→(:Account)
transfers.transfer_id, occurred_at, amount_rwf	properties on TRANSFER (long, long, int); one relationship per payment, so repeated payments are kept

3.2 Import

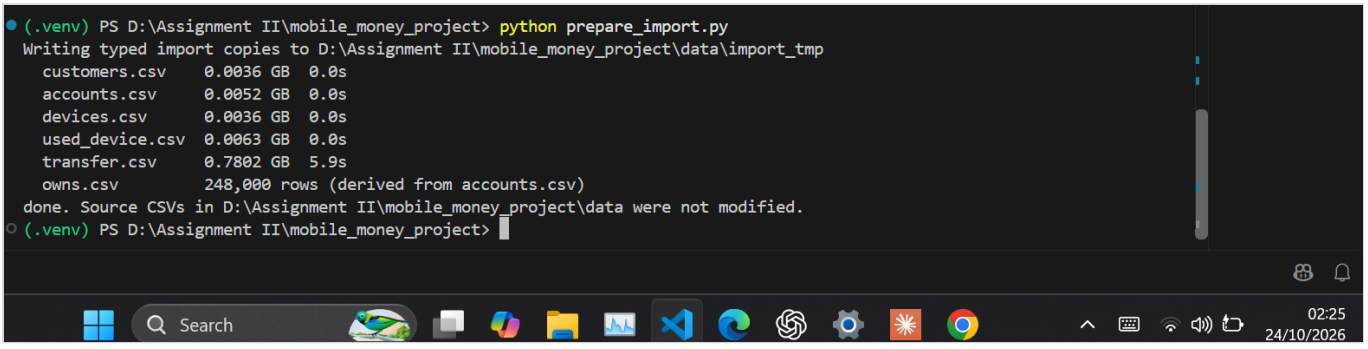


Fig 3.2 — `prepare_import.py`: typed copies written to `data\import_tmp`, owns.csv derived (248,000 rows), source CSVs untouched.

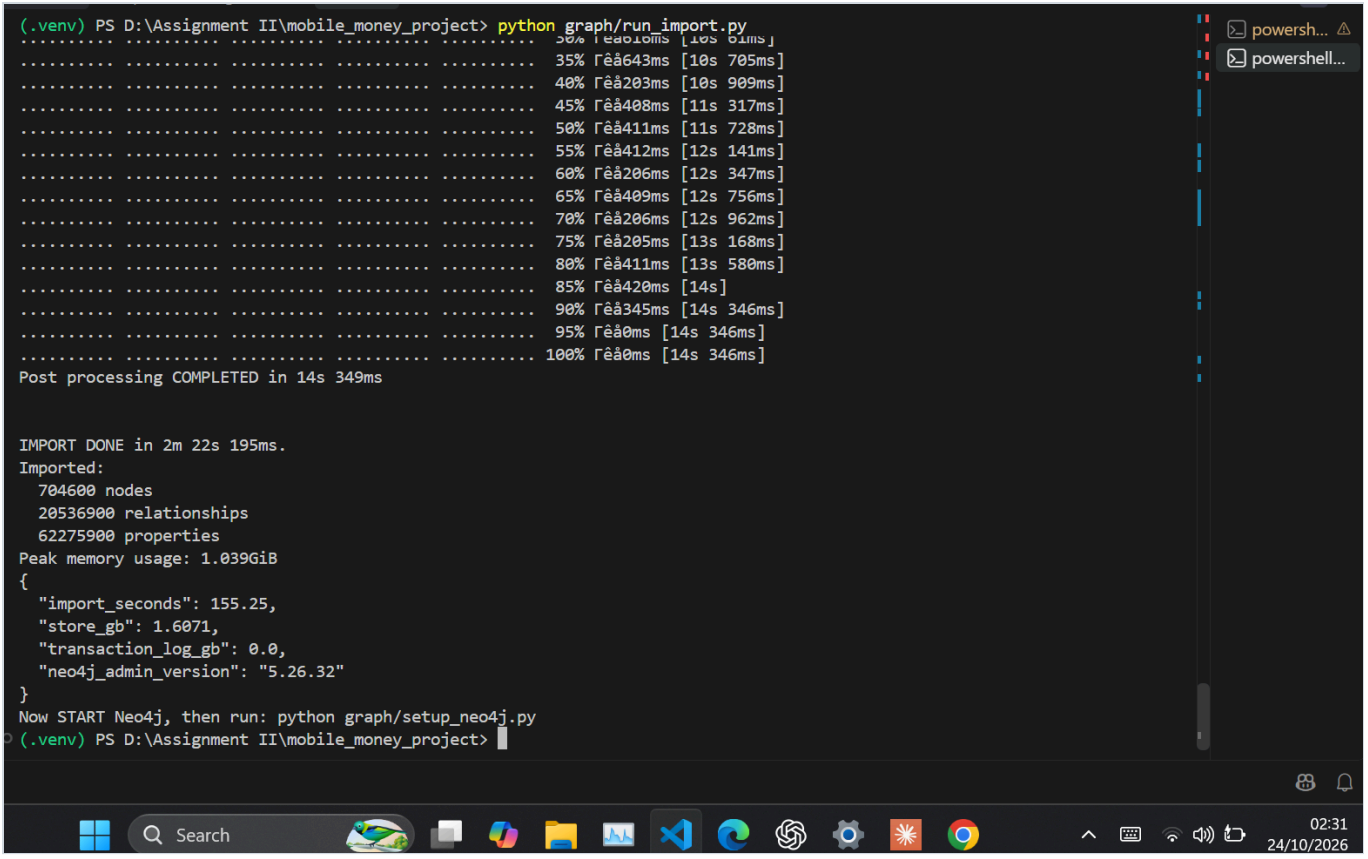


Fig 3.3 — `graph/run_import.py` (Neo4j stopped): import done in 2m 22s; 704,600 nodes, 20,536,900 relationships; script timing 155.25 s; store 1.6071 GB; transaction log 0.0 GB.

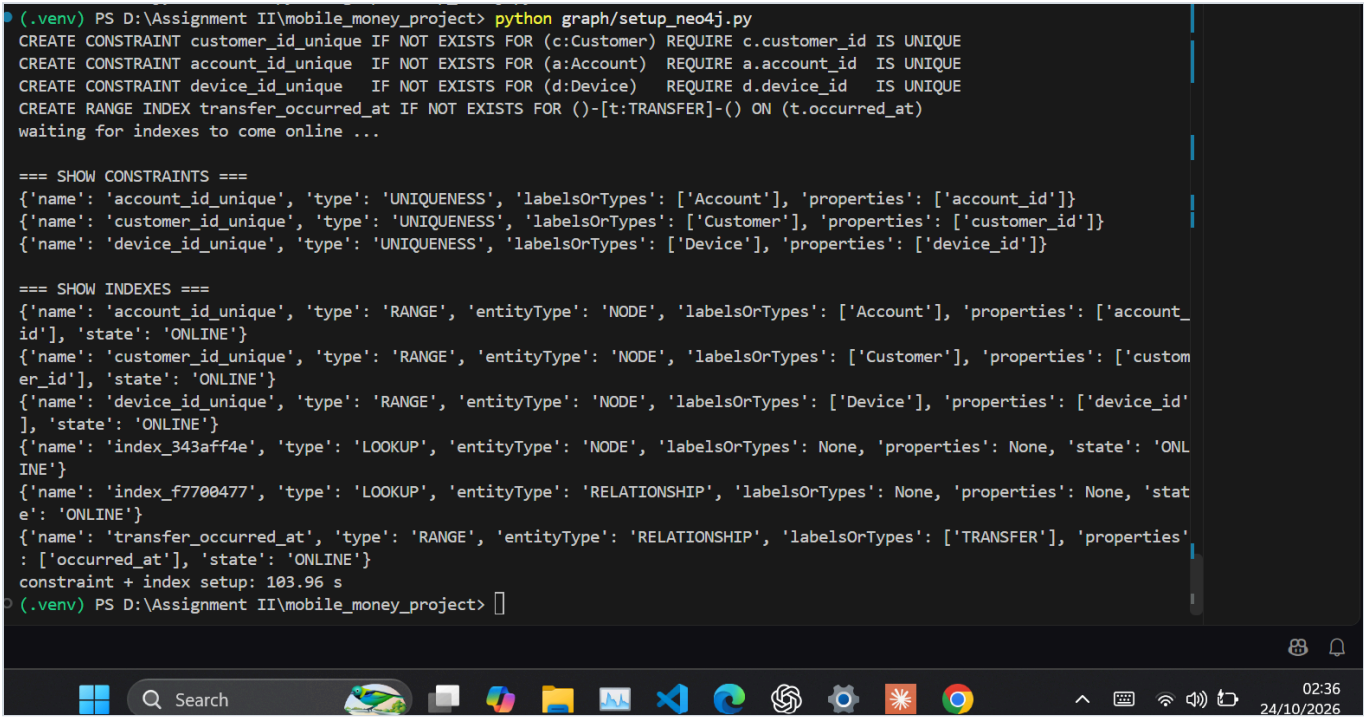


Fig 3.4 — `graph/setup_neo4j.py`: three uniqueness constraints and the range index on TRANSFER.occurred_at, all ONLINE; setup 103.96 s.

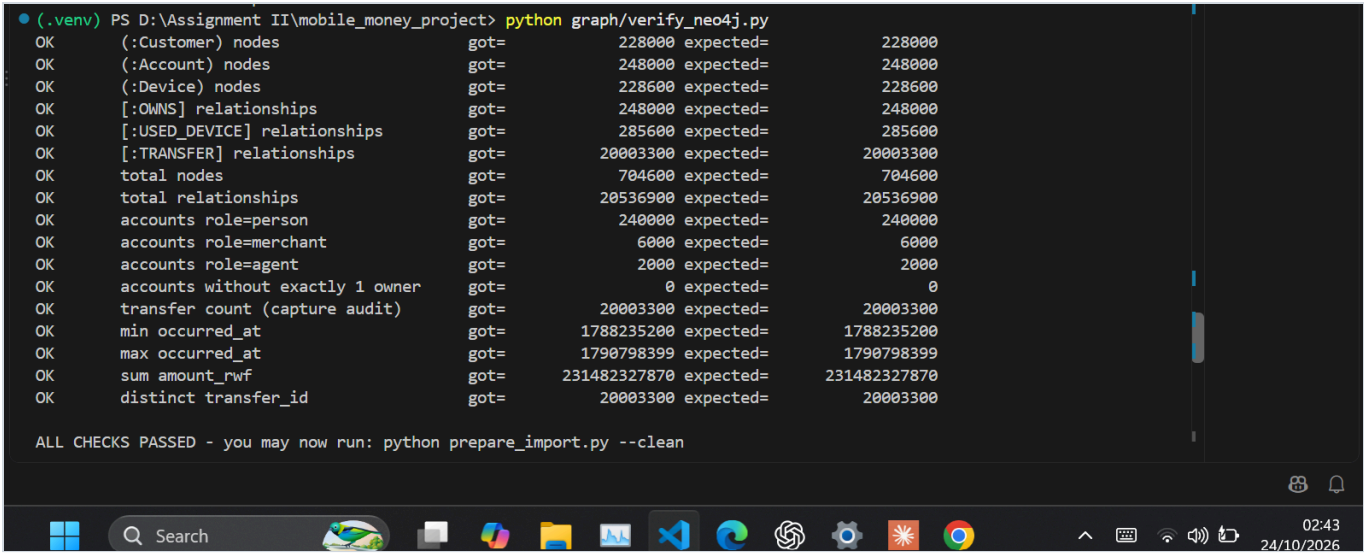


Fig 3.5 — `graph/verify_neo4j.py`: node/relationship counts, roles, one owner per account, unique transfer_id and audit totals all OK.

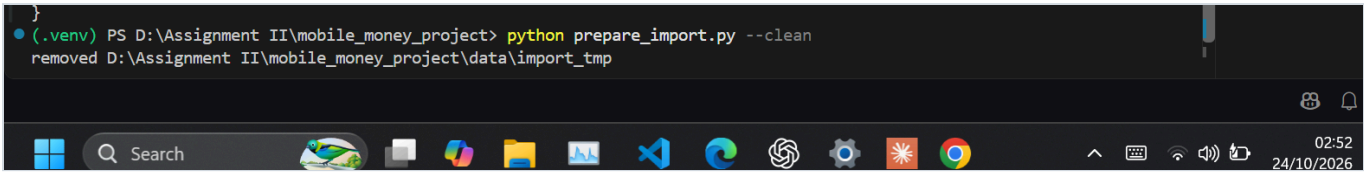


Fig 3.6 — `prepare_import.py --clean` removes the temporary import copies.

Measure	PostgreSQL	Neo4j
Load / import	109.69 s	155.25 s
Constraints / indexes	170.83 s	103.96 s
Total	280.52 s	259.21 s
Stored size	3.7266 GB (tables + indexes)	1.6071 GB (store); transaction log 0.0 GB

3.3 One account in both models

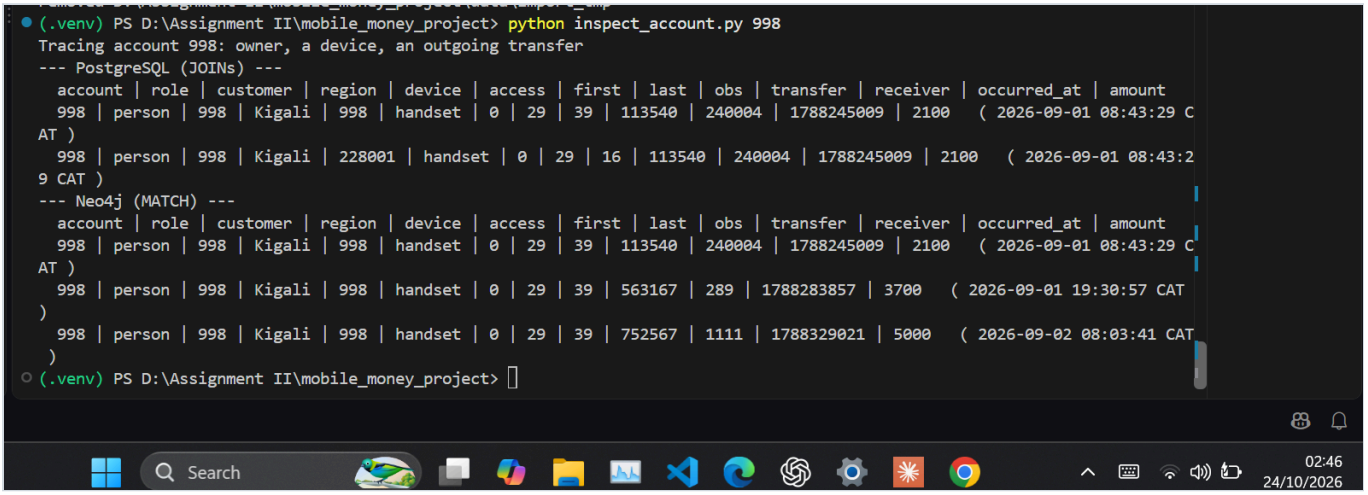


Fig 3.7 — `inspect_account.py 998`: owner (customer 998, Kigali), devices 998 and 228001 (handsets, days 0–29) and an outgoing transfer: #113540 to account 240004, 2,100 RWF at 2026-09-01 08:43:29 CAT, identical in the SQL JOIN and Cypher MATCH output.

Foreign keys become relationships. `accounts.customer_id` becomes an OWNS relationship, each `account_devices` row becomes a USED_DEVICE relationship that keeps its three properties, and each `transfers` row becomes one TRANSFER relationship between the sender and receiver nodes. The engines

return an account's other rows in a different order (Neo4j lists two further transfers), so the first matching row is the evidence.

What a collapsed edge would lose. One edge per account pair would lose every individual `transfer_id`, timestamp and amount. Q1 and Q2 (counts and totals per hour/day), Q5 (10-minute gaps and 90–105% amount rule) and Q6 (increasing time, 1,800-second span) all need those events, so they could not be answered.

4 Routine business questions

Query pairs: `queries/q1..q7.sql / .cypher`. Window: 09:45:00 inclusive to 10:45:00 exclusive CAT on the set's date, converted to epoch seconds in Python (`config.hour_window`).

4.1 Q1 Account activity

Set	Account	Date (Sep)	Outgoing count	Total RWF
A	998	6	1	200,000
B	147228	13	1	1,050,000
C	206978	8	1	300,000
D	237998	6	1	45,000
E	144228	13	1	40,000
F	203978	8	1	30,000

Values from the 61-case comparison (identical in both engines and the CSV reference).

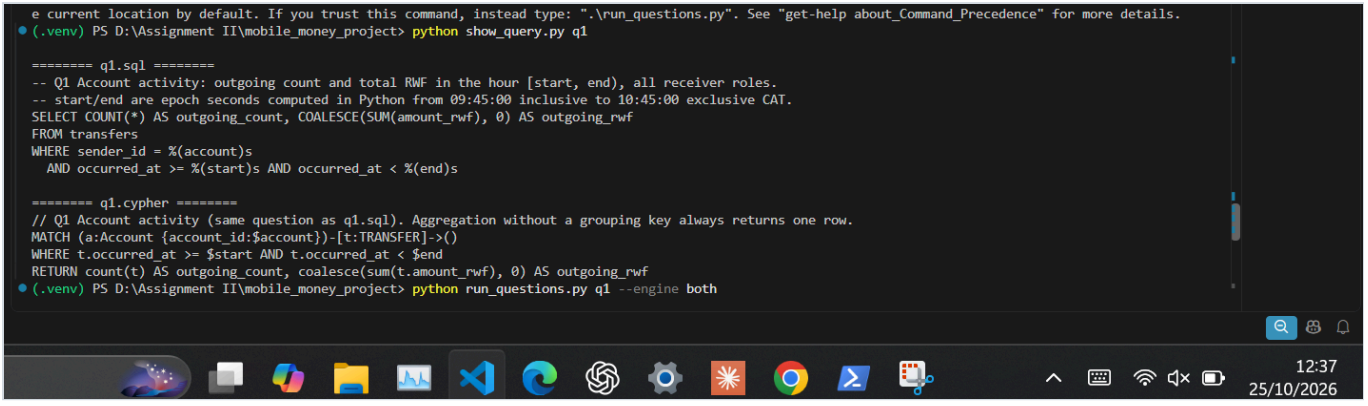


Fig 4.1 — `show_query.py q1`: the parameterized SQL and Cypher pair (start/end epoch seconds computed in Python; COALESCE gives 0 when nothing matches).

Why outgoing value is not a balance. It counts only money leaving in one hour. It ignores incoming payments, opening balance, cash-in and cash-out at agents, and fees, so a wallet can send 1,050,000 RWF and still hold little or much.

4.2 Q2 Daily activity

30 CAT days returned by both engines; totals reconcile to the capture audit: 20,003,300 transfers and 231,482,327,870 RWF (first day 666,667 transfers, 7,683,326,000 RWF; last day 666,666 and 7,684,945,400 RWF).

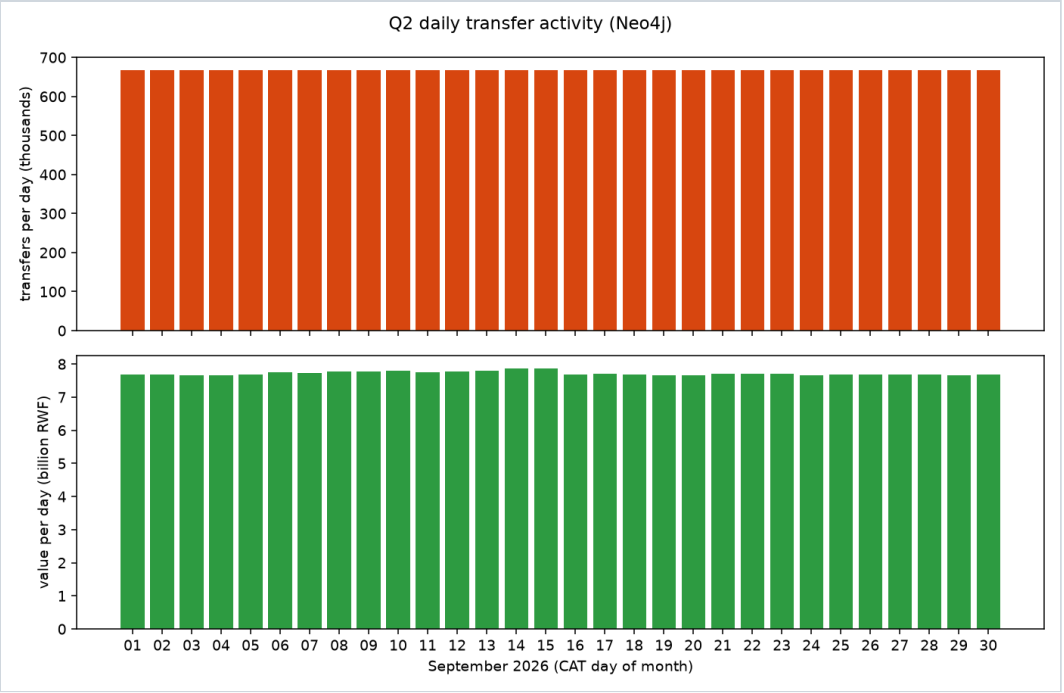


Fig 4.2 — Daily transfers (thousands) and value (billion RWF) per CAT day, 1–30 September 2026.

Pattern. Daily count is almost constant (666,666–666,997), but value is higher on 6–15 September (counts +0.04%, value up to +2.3%), peaking on 15 Sep at 7.86 billion RWF against a 7.72 billion average, so the average payment is larger on those days. **Not revenue:** transfer value is money moved on behalf of customers; the provider earns only fees or commissions, which these records do not contain.

4.3 Q3 Shared devices

Set	Rows	Other accounts (device)
A (998)	4	16836, 24755, 8917 share 228001; 220998 shares 998
B (147228)	9	147226, 147227, 147229, 147230 (device 147226); 155147, 163066, 170985, 178904, 186823 (228018)
C (206978)	9	206976, 206977, 206979, 206980 (206976); 214897, 222816, 230735, 238654, 6573 (228043)
D (237998)	4	13836, 21755, 5917 share 228301; 17998 shares 17998
E (144228)	9	144226…144230 (144226); 152147, 160066, 167985, 175904, 183823 (228318)
F (203978)	9	203976…203980 (203976); 211897, 219816, 227735, 235654, 3573 (228343)

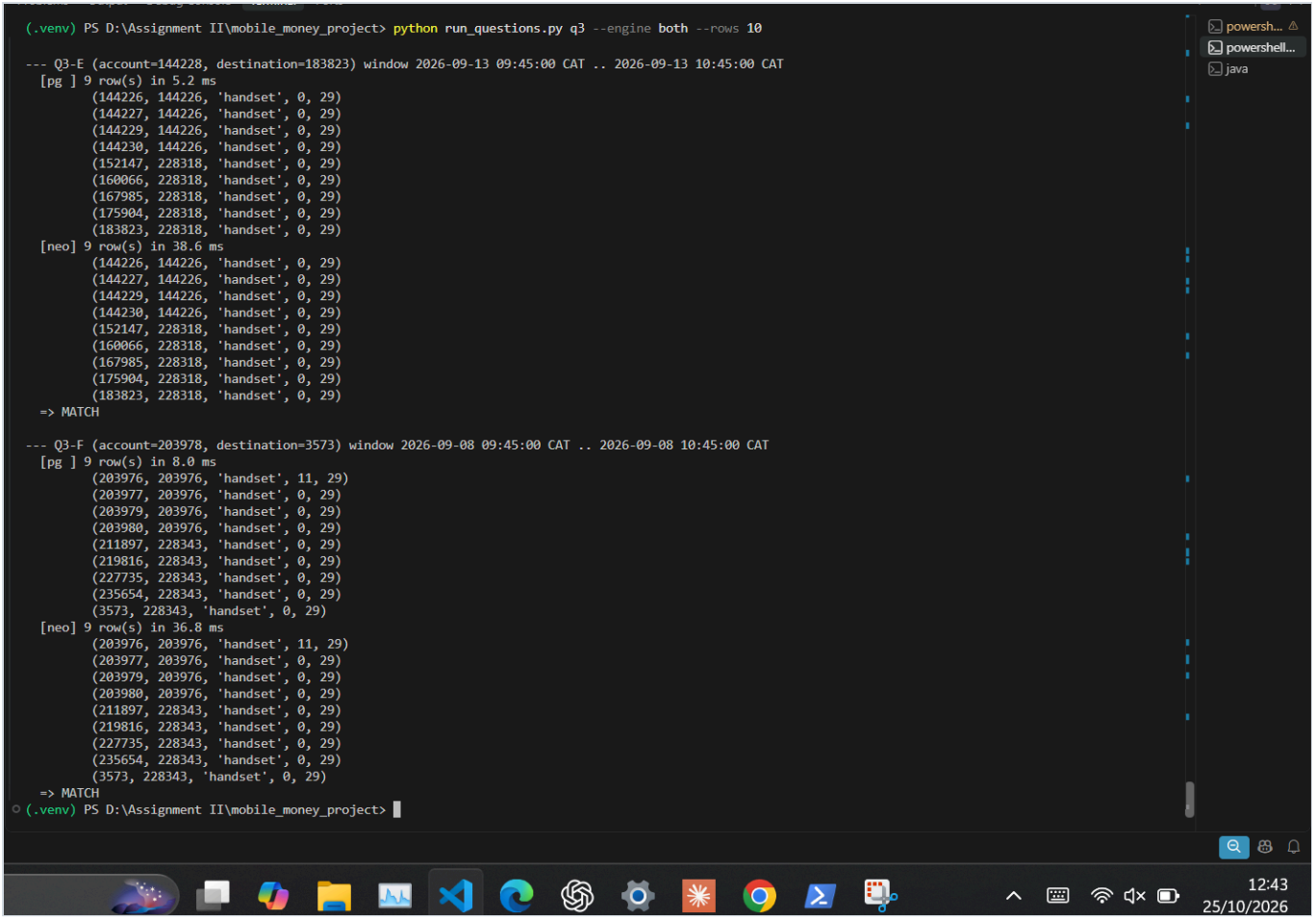


Fig 4.3 — `run_questions.py q3 --engine both --rows 10`: sets E and F, PostgreSQL and Neo4j rows identical (MATCH); all six sets are in the table above.

Legitimate sharing. A family handset used by relatives, or one shop phone used by several staff, appears as many accounts on one device. **Overlap is not simultaneous use:** the data store only first/last day and a count. Intervals [2,7] and [5,11] overlap on days 5–7, but one account may have used the phone in the morning and the other at night, or on different days within that range.

5 Connected accounts

5.1 Q4 Ownership and device network

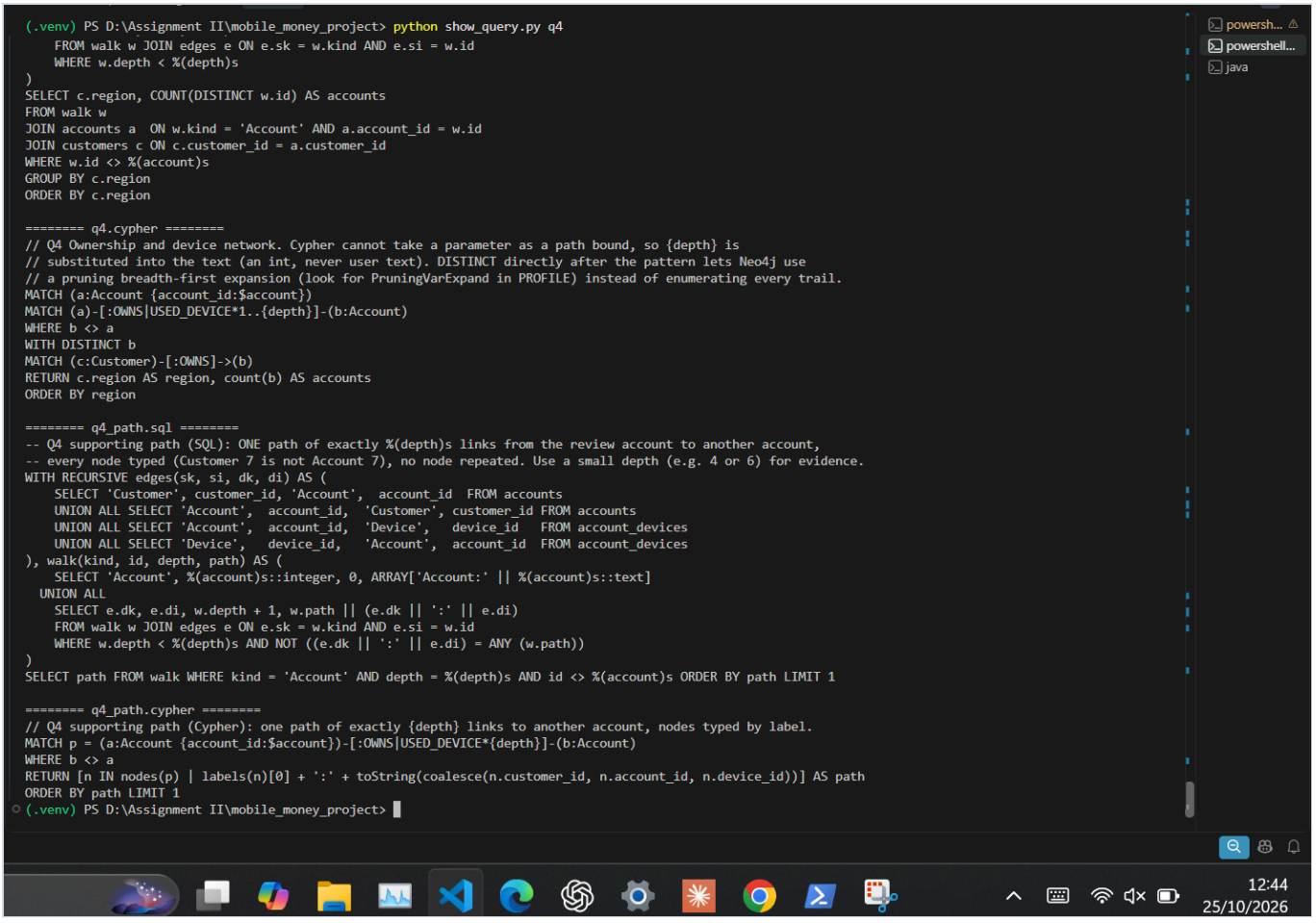


Fig 5.1 — `show_query.py q4`: end of the recursive SQL (walk over typed edges, distinct accounts by region), the Cypher pattern with DISTINCT before grouping, and the two supporting-path queries.

Set	Depth 4	Depth 6	Depth 8	Depth 10
A	10	43	175	187
B	46	178	188	228
C	146	483	832	1,433
D	10	44	180	192
E	47	183	193	230
F	147	488	837	1,426

Distinct other accounts reached (all regions); identical in PostgreSQL, Neo4j and the reference for all 24 cases.

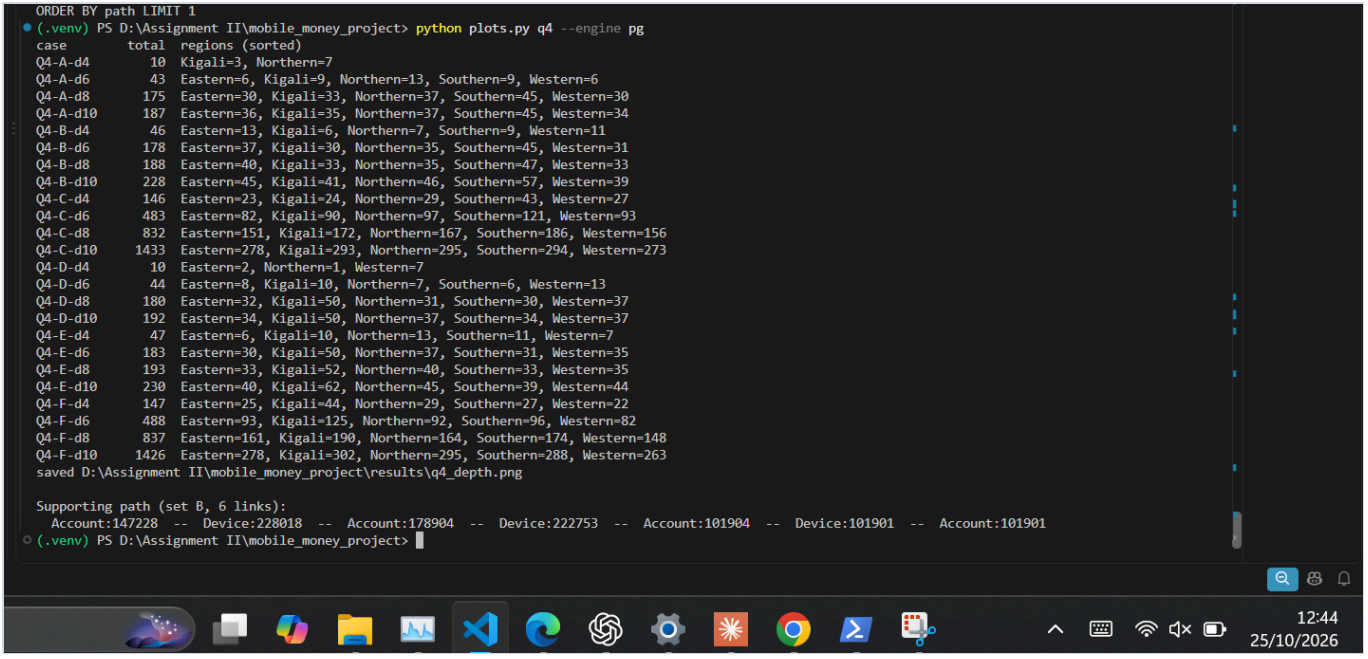


Fig 5.2 — `plots.py q4 --engine pg`: all 24 cases with regional counts and one supporting path (set B, 6 links): account 147228 – device 228018 – account 178904 – device 222753 – account 101904 – device 101901 – account 101901.

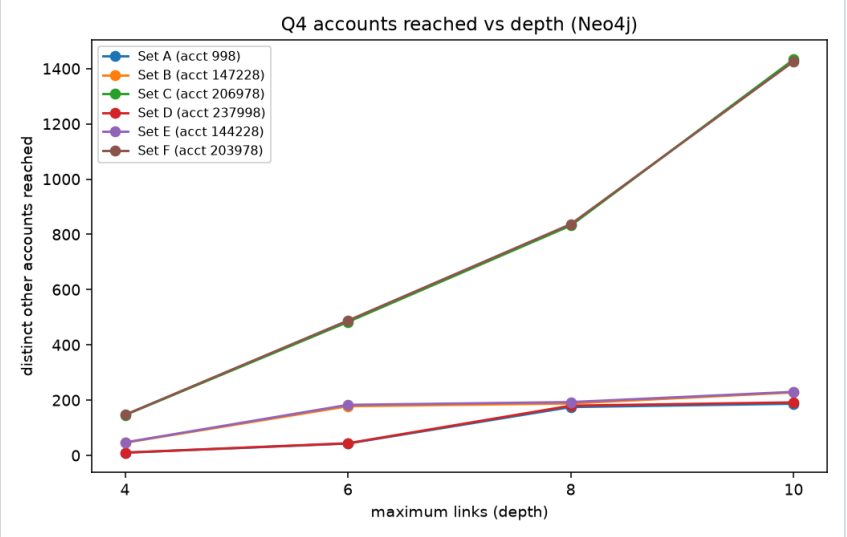


Fig 5.3 — Accounts reached versus depth for the six sets.

> **Predictions for depths 8 and 10.** The group did not write predictions down before running depths 8 and 10, so no prediction-versus-result comparison is claimed.

Weak associations. Every extra link multiplies the reachable set (A: 10 accounts at depth 4, 187 at depth 10; C: 146 to 1,433). At depth 10 an account is linked to hundreds of people through chains such as owner–account–shared handset–account–owner, and one shared device is enough to join unrelated households. Large counts therefore mean “weakly connected”, not “suspicious”. Sets C and F reach far more accounts at every depth, yet differ only slightly from each other (1,433 vs 1,426), so network size does not separate a pair.

5.2 Q5 Rapid onward payments

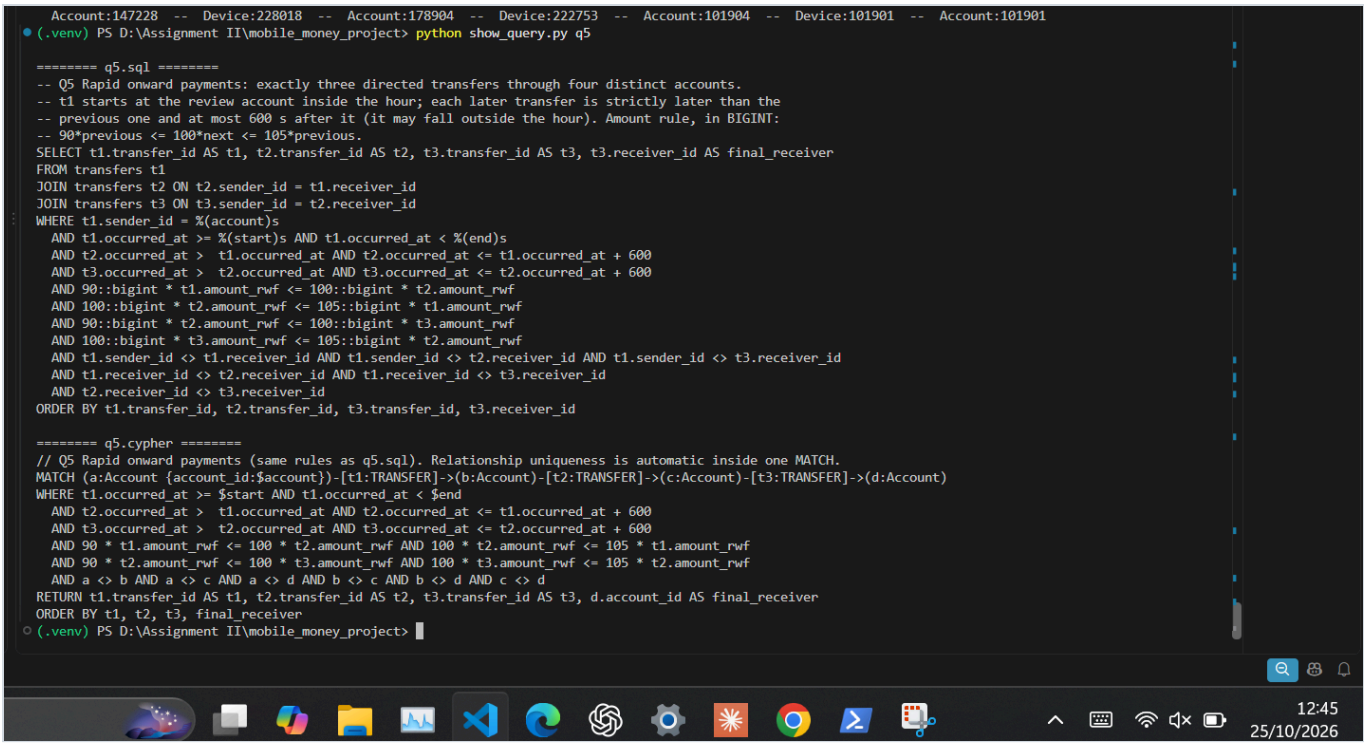


Fig 5.4 — `show_query.py q5`: SQL (three transfer aliases, BIGINT amount rule) and Cypher (three directed relationships).

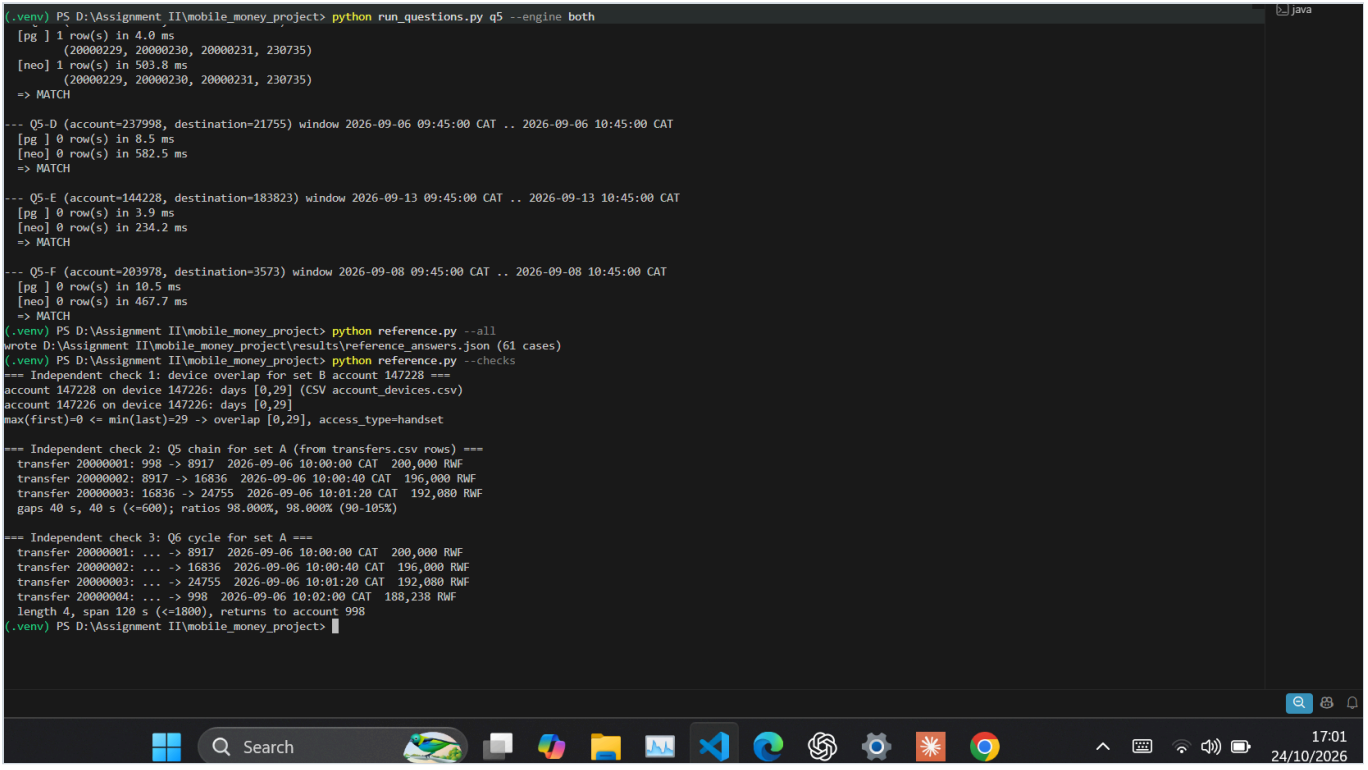


Fig 5.5 — `run_questions.py q5 --engine both` (sets C–F visible, all MATCH), then `reference.py --checks`: independent chain check for set A from the CSV rows (IDs, CAT times, amounts, gaps 40 s and 40 s, ratios 98.000%).

Set	Chains	Transfer IDs t1, t2, t3	Final receiver
A	1	20000001, 20000002, 20000003	24755
B	1	20000092, 20000093, 20000094	170985
C	1	20000229, 20000230, 20000231	230735
D	0	–	–
E	0	–	–
F	0	–	–

Transfer	Sender → receiver	CAT time (6 Sep)	Amount RWF	% of previous
20000001	998 → 8917	10:00:00	200,000	–
20000002	8917 → 16836	10:00:40	196,000	98.0%
20000003	16836 → 24755	10:01:20	192,080	98.0%

Why this does not prove the same funds moved. Money is fungible: an account may receive and pay amounts of similar size for unrelated reasons (bill splitting, a shop passing on a customer's payment, a fee-bearing relay). The rule tests only timing and amounts, not whether the received money was used for the onward payment.

5.3 Q6 Circular payments

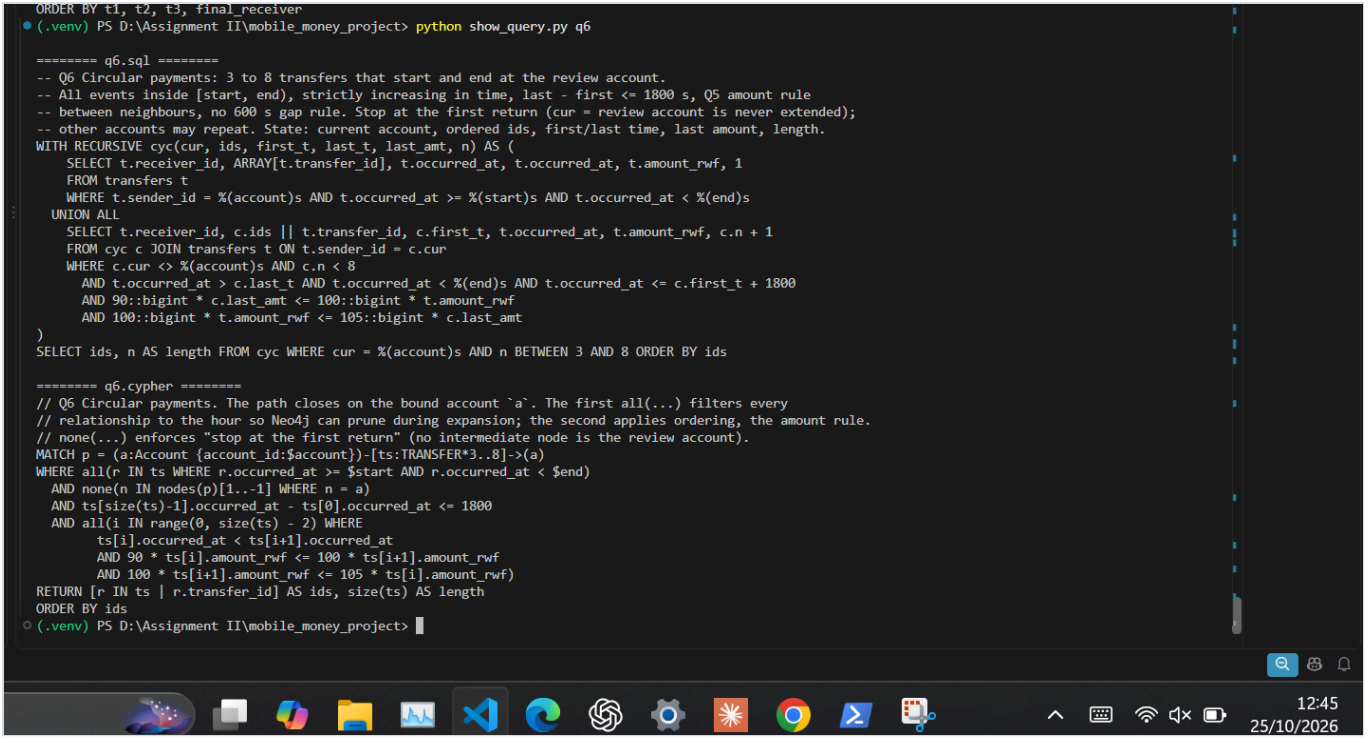


Fig 5.6 — `show_query.py q6`: recursive SQL carrying current account, ordered IDs, first/last time and last amount, and the bounded Cypher path with equivalent checks.

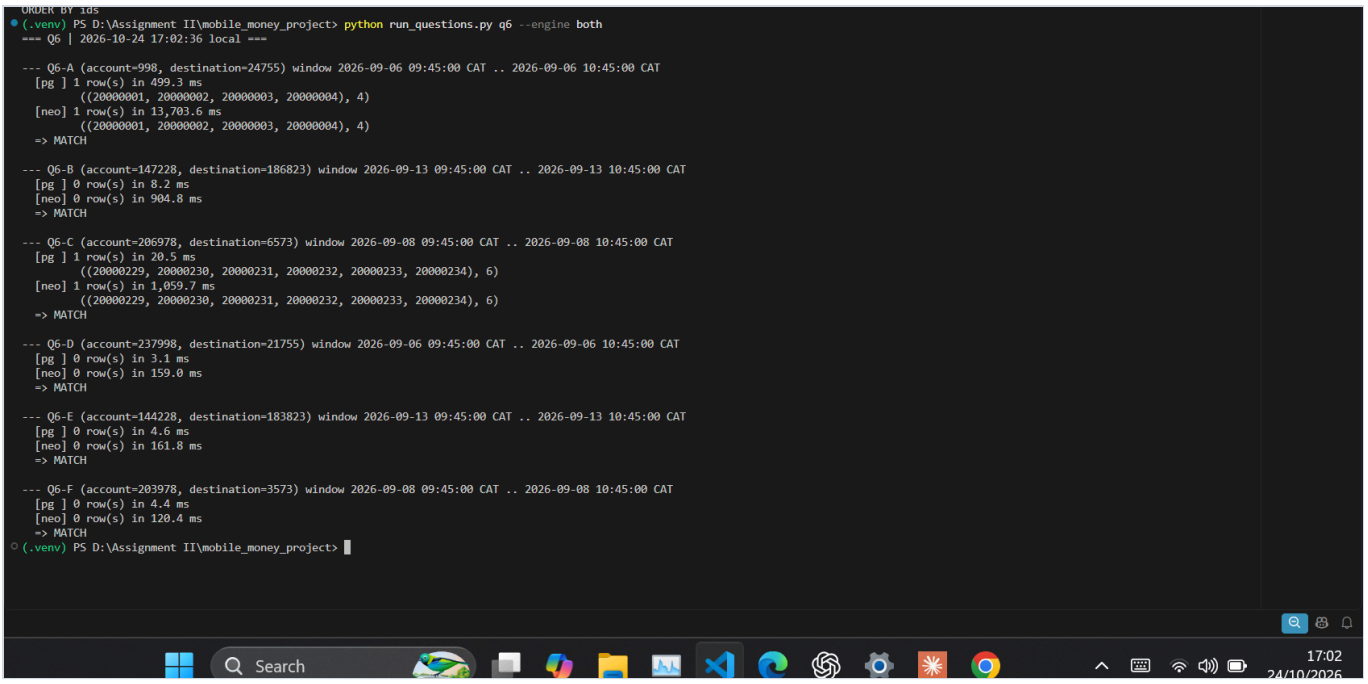


Fig 5.7 — `run_questions.py q6 --engine both`: six sets, both engines, all MATCH. A returns one cycle of length 4, C one of length 6; B, D, E, F none.

Set	Cycles	Transfer-ID sequence	Length
A	1	20000001, 20000002, 20000003, 20000004	4
C	1	20000229 … 20000234	6
B, D, E, F	0	–	–

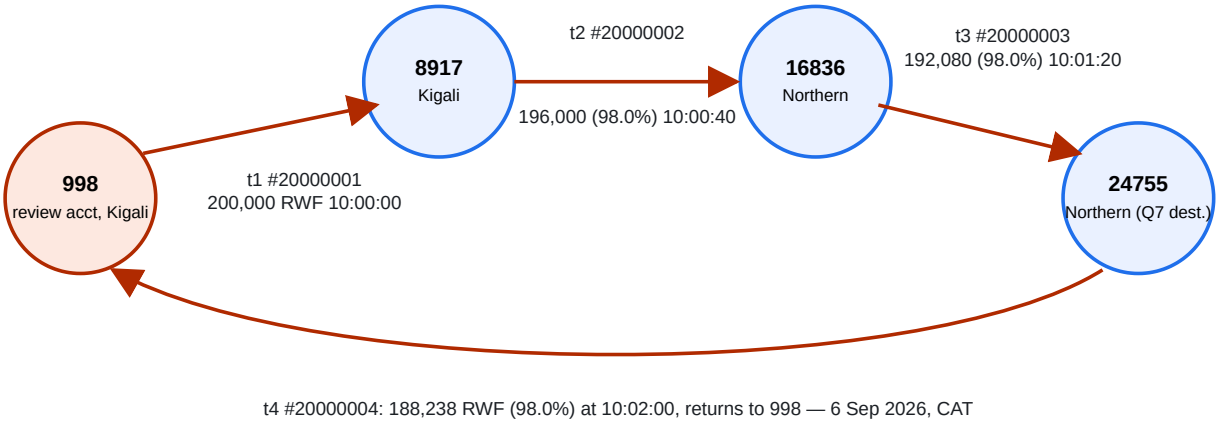


Fig 5.8 — Set A cycle (also the Q5 chain): four transfers in 120 seconds, each 98.0% of the previous, returning to the starting account (verified from CSV rows in Fig 5.5).

Further evidence to request: which device initiated each payment, the identity and relationship of the four owners, the stated purpose of each payment, the account history before September (is this pattern repeated?), and the tariff (a 2% reduction per hop could simply be a fee).

5.4 Q7 Shortest transfer connection

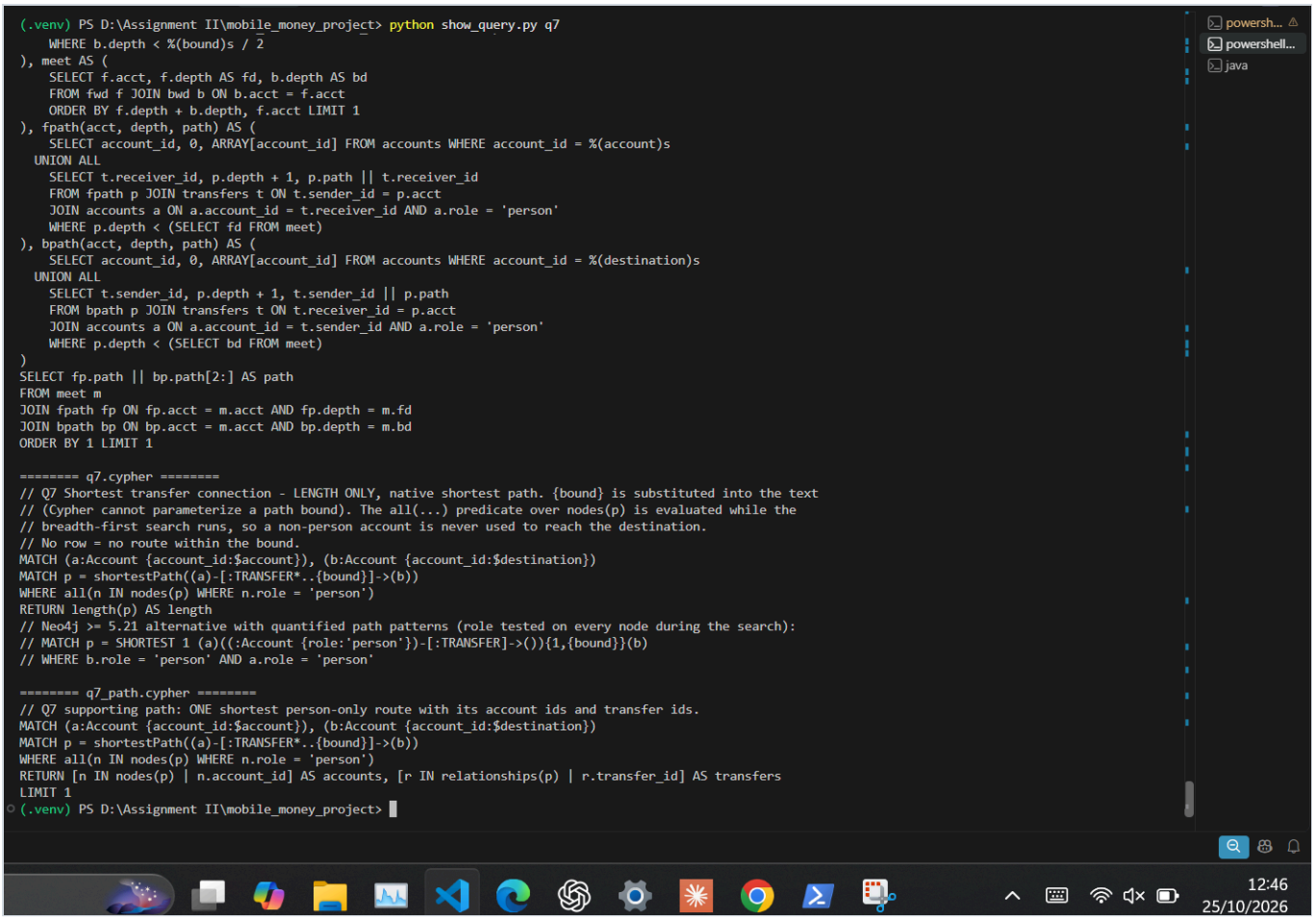


Fig 5.9 — `show_query.py q7`: end of the SQL path query, the Cypher `shortestPath` with the person predicate, and the supporting-path Cypher.

Set	Bound 4	Bound 6	Supporting path (checked in both engines)
A	3	3	998, 8917, 16836, 24755
B	none	5	PG: 147228, 147793, 147197, 148101, 178904, 186823; Neo4j: 147228, 155147, 163066, 170985, 178904, 186823 (equally short)
C	none	5	valid
D	3	3	valid
E	none	5	valid
F	none	5	valid

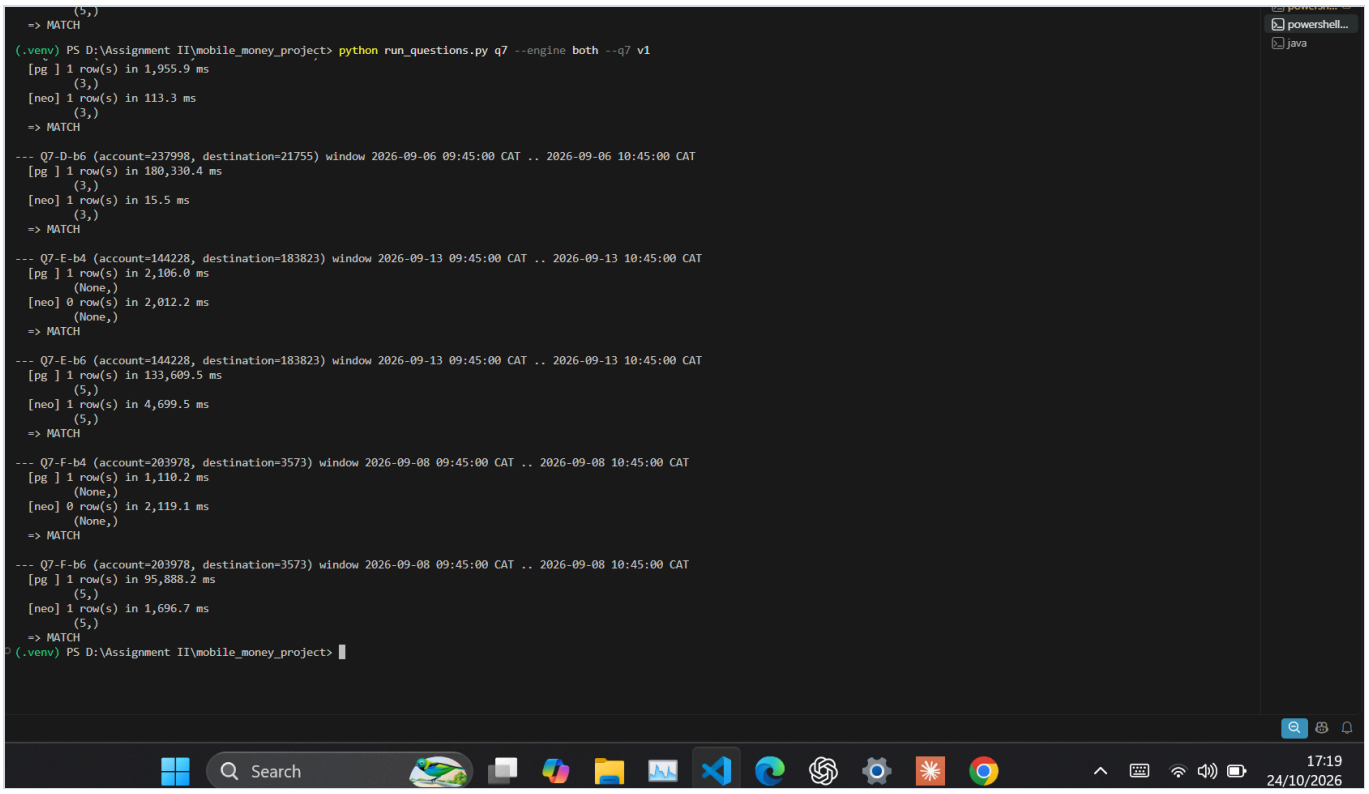


Fig 5.10 — `run_questions.py q7 --engine both --q7 v1` (sets D–F visible): lengths match; baseline PostgreSQL bound-6 times of 96–180 s occur in this correctness run, before the timed benchmark.

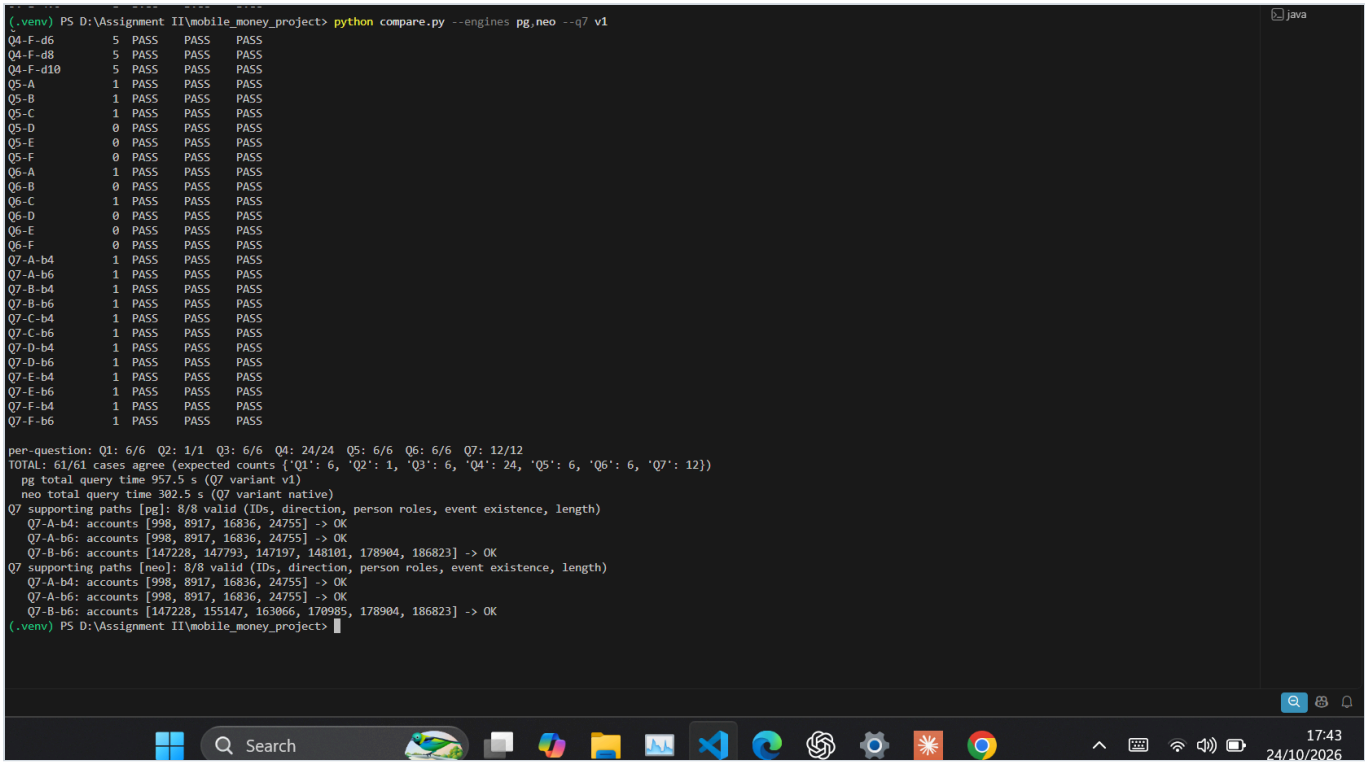


Fig 5.11 — 61-case table with the baseline Q7 (first run, pg=neo=reference) and the Q7 path validation: 8/8 valid in each engine; equally short paths differ (set B).

No route within four does not mean no connection: B, C, E and F have routes of five transfers; a longer route, a route through a merchant or agent, ownership/device links, or payments outside September are all excluded by the rules. **A structural route need not show movement of funds:** amounts and time order are ignored, so the transfers may run backwards in time or involve unrelated sums.

6 Correctness and performance

6.1 One comparison program

`compare.py` runs 61 cases (Q1 6, Q2 1, Q3 6, Q4 24, Q5 6, Q6 6, Q7 12) through `run_pg` and `run_neo`, normalises integers, sorts rows and compares exactly with each other and with `reference.py`, which recomputes answers from the CSV files. Q7 lengths are compared separately from supporting paths. Result: **61/61 agree** (Fig 5.11; full log in `results/`).

Independent checks from CSV rows: device overlap for account 147228 and 147226 on device 147226, days [0,29], (Fig 5.5), the Q5 chain for A (gaps 40 s, ratios 98%) and the Q6 cycle for A (length 4, span 120 s, returns to 998).

6.2 Timing a fixed selection

Same laptop, one database service at a time, persistent connections, default PostgreSQL settings and Neo4j heap 3.52 GiB / page cache 1.95 GiB. Three untimed warm-ups (180 s limit) and five measured runs (60 s limit); timeouts are recorded separately and never as timings. Ratio = PostgreSQL baseline median / Neo4j median; the last two columns use the improved Q7 (v2).

Case	PG median ms	PG min	PG max	Neo4j median ms	Neo min	Neo max	PG/Neo	PG v2 median	v2/Neo
Q1-B	1.2	1.1	1.4	22.1	19.1	47.4	0.055	–	–
Q1-E	1.2	1.2	1.5	14.9	9.0	18.2	0.081	–	–
Q2	13,059	11,258	13,645	timeout (5/5)	–	–	–	–	–
Q3-B	2.0	1.4	2.7	40.2	29.0	75.7	0.051	–	–
Q3-E	2.1	1.5	2.3	49.6	40.7	67.4	0.042	–	–
Q4-B-d4	4.2	3.3	6.4	49.1	35.4	54.0	0.085	–	–
Q4-B-d6	11.0	9.8	13.0	123.2	35.5	145.5	0.089	–	–
Q4-B-d8	12.4	10.1	15.8	58.1	32.3	97.4	0.213	–	–
Q4-B-d10	21.4	20.6	25.8	34.3	28.7	56.5	0.625	–	–
Q4-E-d4	4.5	2.3	5.8	26.7	19.5	47.8	0.17	–	–
Q4-E-d6	7.0	5.3	9.2	66.9	36.4	91.0	0.104	–	–
Q4-E-d8	12.2	10.4	13.8	60.6	50.7	132.5	0.201	–	–
Q4-E-d10	18.0	14.3	22.8	34.0	25.9	68.7	0.529	–	–
Q5-B	3.0	2.0	6.5	42.7	17.9	52.6	0.07	–	–
Q5-E	2.2	1.6	4.1	21.4	17.1	67.7	0.102	–	–
Q6-B	1.7	1.3	3.3	53.3	20.9	85.2	0.031	–	–
Q6-E	2.0	1.6	2.7	23.5	10.2	34.1	0.086	–	–
Q7-B-b4	380.0	354.4	402.9	34.0	23.8	82.1	11.168	10.1	0.297
Q7-B-b6	40,731 [a]	40,731	40,731	54.9	40.8	68.6	741.78	94.9	1.729
Q7-E-b4	450.3	419.4	634.2	35.6	18.1	40.1	12.663	11.1	0.313
Q7-E-b6	timeout (5/5)	–	–	66.1	62.0	84.6	–	64.6	0.977

[a] Q7-B-b6 baseline: only one of five measured runs completed (40,731 ms); the other four timed out at 60 s, so min = max = median is a single value. Q2 in Neo4j and Q7-E-b6 baseline completed 0 of 5 runs.

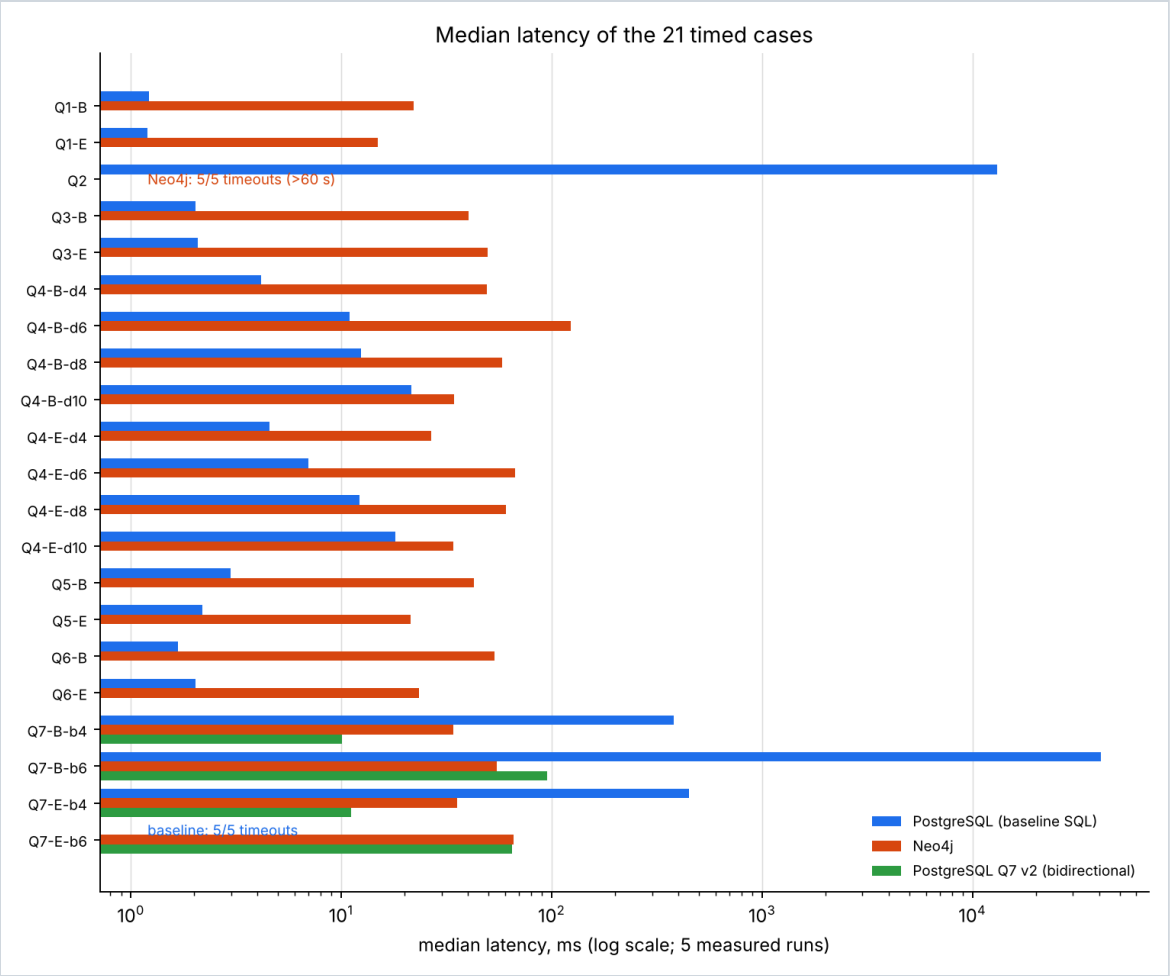
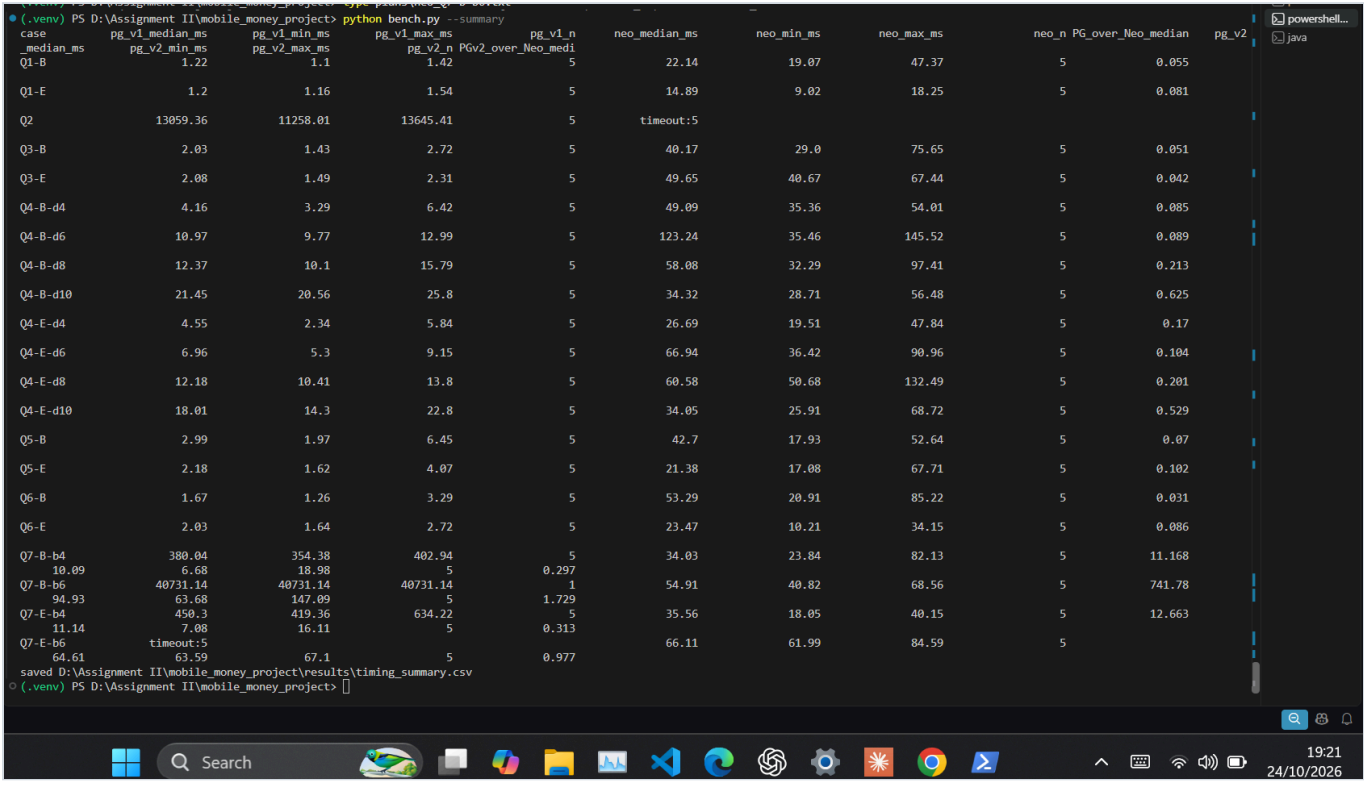


Fig 6.1 — Median latency per case (log scale).



```
(.venv) PS D:\Assignment II\mobile_money_project> python bench.py --summary
case          pg_v1_median_ms      pg_v1_min_ms      pg_v1_max_ms          pg_v1_n    neo_median_ms     neo_min_ms     neo_max_ms          neo_n PG_over_Neo_median    pg_v2
_median_ms      pg_v2_min_ms      pg_v2_max_ms          pg_v2_n PGv2_over_Neo_medi
Q1-B                    1.22               1.1              1.42                5            22.14          19.07          47.37              5               0.055
Q1-E                     1.2              1.16              1.54                5            14.89           9.02          18.25              5               0.081
Q2                   13059.36          11258.01          13645.41                5         timeout:5
Q3-B                    2.03               1.43              2.72                5            40.17           29.0          75.65              5               0.051
Q3-E                    2.08               1.49              2.31                5            49.65          40.67          67.44              5               0.042
Q4-B-d4                 4.16               3.29              6.42                5            49.09          35.36          54.01              5               0.085
Q4-B-d6                10.97               9.77             12.99                5           123.24          35.46         145.52              5               0.089
Q4-B-d8                12.37               10.1             15.79                5            58.08          32.29          97.41              5               0.213
Q4-B-d10               21.45              20.56              25.8                5            34.32          28.71          56.48              5               0.625
Q4-E-d4                 4.55               2.34              5.84                5            26.69          19.51          47.84              5                0.17
Q4-E-d6                 6.96                5.3              9.15                5            66.94          36.42          90.96              5               0.104
Q4-E-d8                12.18              10.41              13.8                5            60.58          50.68         132.49              5               0.201
Q4-E-d10               18.01               14.3              22.8                5            34.05          25.91          68.72              5               0.529
Q5-B                    2.99               1.97              6.45                5             42.7          17.93          52.64              5                0.07
Q5-E                    2.18               1.62              4.07                5            21.38          17.08          67.71              5               0.102
Q6-B                    1.67               1.26              3.29                5            53.29          20.91          85.22              5               0.031
Q6-E                    2.03               1.64              2.72                5            23.47          10.21          34.15              5               0.086
Q7-B-b4               380.04             354.38            402.94                5            34.03          23.84          82.13              5              11.168
     10.09               6.68              18.98                5            0.297
Q7-B-b6             40731.14           40731.14          40731.14                1            54.91          40.82          68.56              5              741.78
     94.93              63.68             147.09                5            1.729
Q7-E-b4                450.3             419.36            634.22                5            35.56          18.05          40.15              5              12.663
     11.14               7.08              16.11                5            0.313
Q7-E-b6             timeout:5                                                              66.11          61.99          84.59              5
     64.61              63.59              67.1                5            0.977
saved D:\Assignment II\mobile_money_project\results\timing_summary.csv
(.venv) PS D:\Assignment II\mobile_money_project>
```

Fig 6.2 — `bench.py --summary` (median/min/max, PG/Neo ratio).

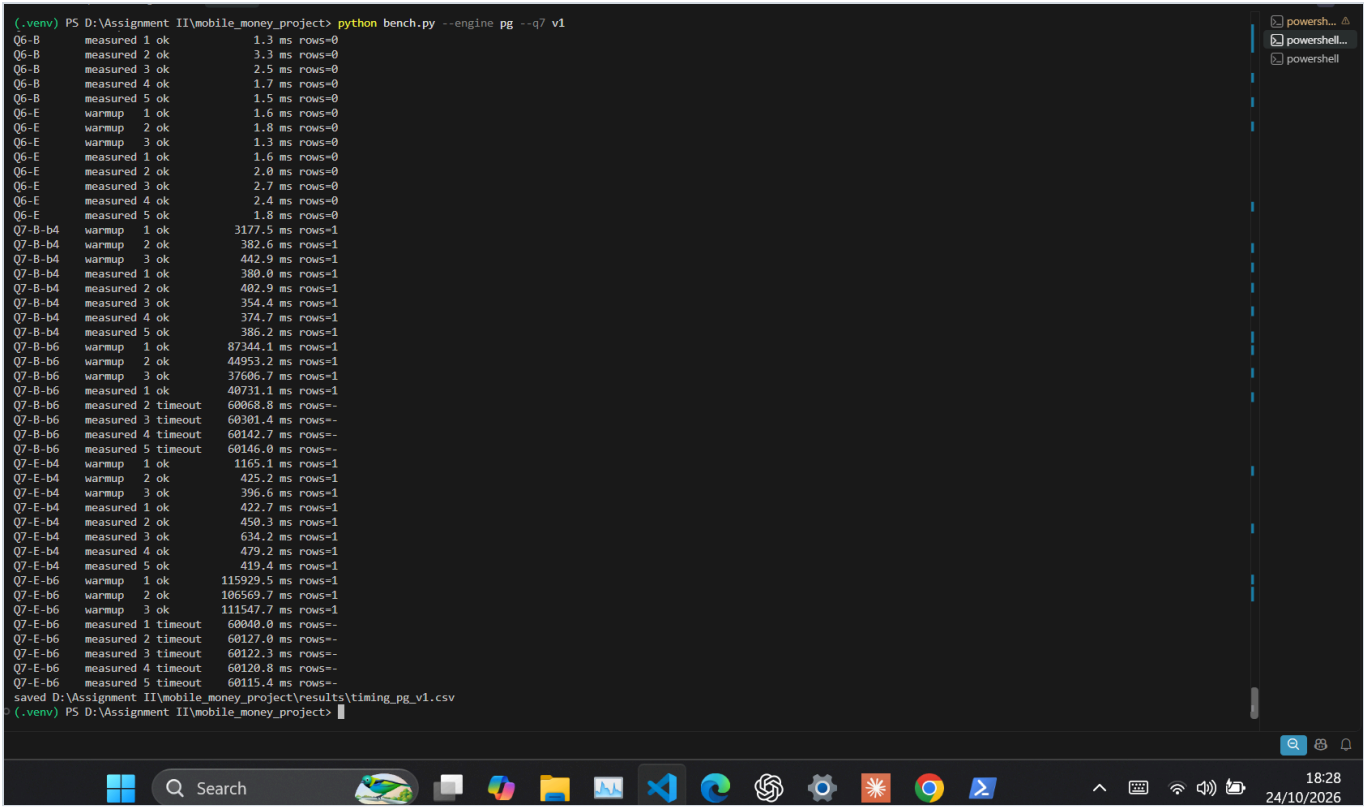


Fig 6.3 — `bench.py --engine pg --q7 v1`: final cases; Q7-B-b6 has one ok run and four timeouts, Q7-E-b6 five timeouts.

Plans (outside timing)

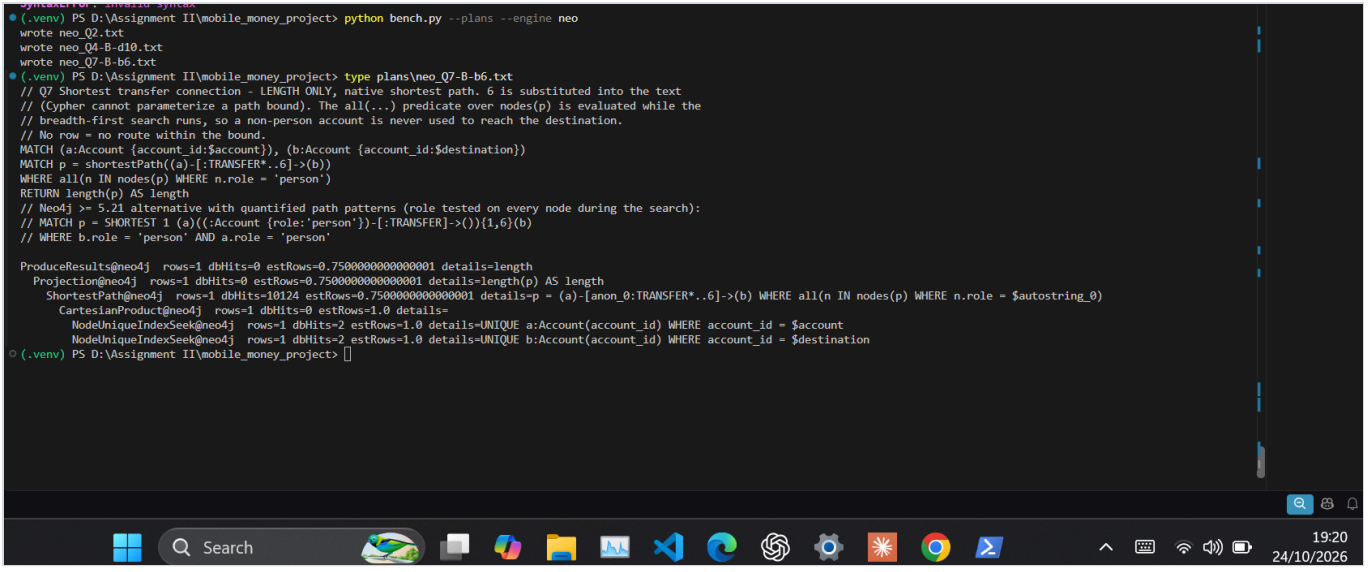


Fig 6.4 — Neo4j PROFILE for Q7-B bound 6 (shortestPath with the all(...) person predicate during the search; 10,124 db hits).

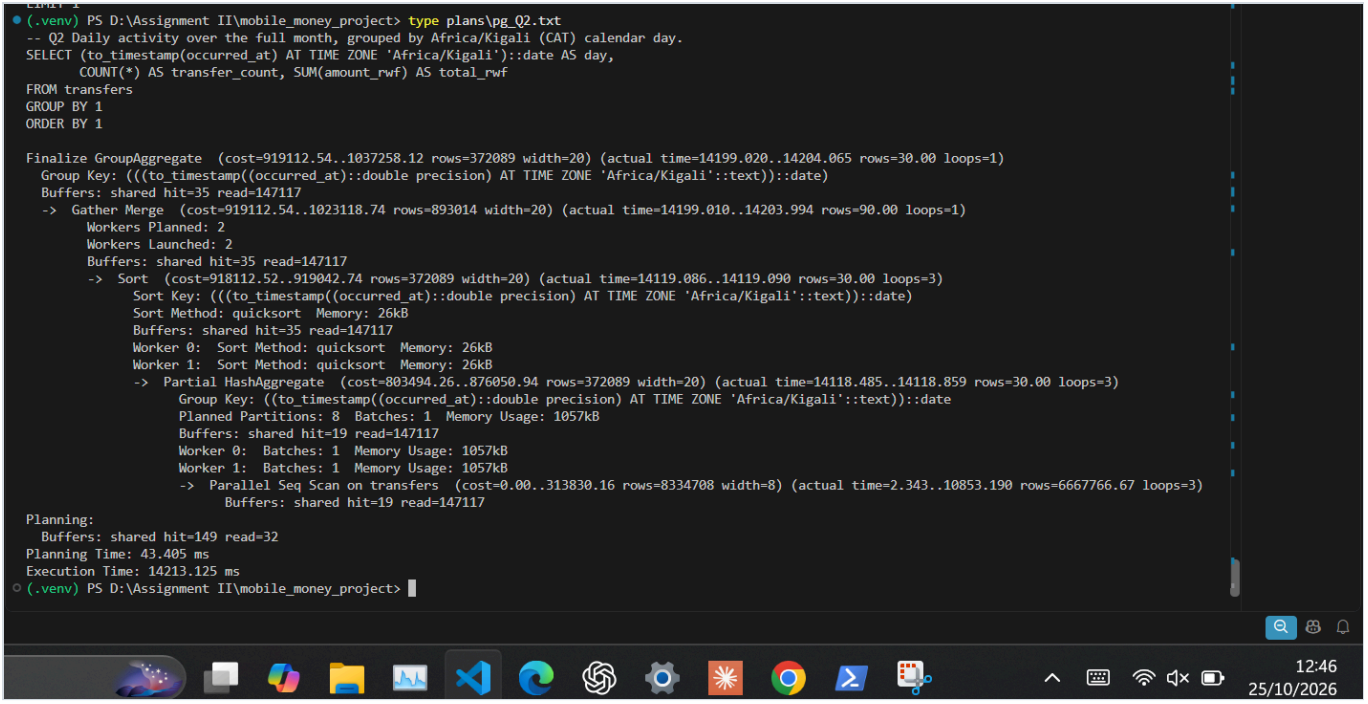


Fig 6.5a — PostgreSQL EXPLAIN (ANALYZE, BUFFERS) for Q2: Parallel Seq Scan (3 processes), partial HashAggregate, 14,213 ms.

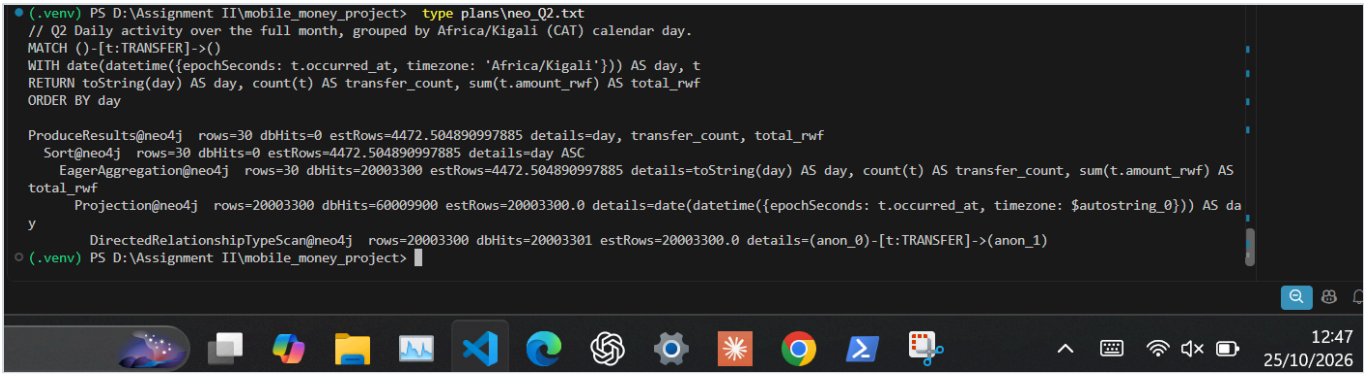


Fig 6.5b — Neo4j PROFILE for Q2: DirectedRelationshipTypeScan of 20,003,300 relationships, Projection with 60,009,900 db hits, EagerAggregation. The Q4 depth-10 plans are in `plans/` in the code submission.

One processing difference (Q2). PostgreSQL's plan is a *Parallel Seq Scan* over 147,117 pages with a partial HashAggregate in each of three processes (14.2 s under ANALYZE). Neo4j's PROFILE shows a *DirectedRelationshipTypeScan* of 20,003,300 TRANSFER relationships followed by a *Projection* that evaluates `date(datetime(...))` for every one (60,009,900 db hits) and an *EagerAggregation*; there is no parallel operator, and all five measured runs exceeded 60 s. The relational scan streams contiguous pages in parallel; the graph engine is built to follow relationships from a few start nodes, not to aggregate every relationship.

Q4 depth 10: PostgreSQL joins a 1,187-row working set to four index paths per frontier node (283.6 ms under ANALYZE, 21 ms warm), Neo4j uses *VarLengthExpand(Pruning, BFS)* that touches 453 nodes with 1,885 db hits.

6.3 One SQL improvement (Q7)

Version 1 (baseline): a recursive CTE with `UNION` on (account, depth). **Waste in the plan (Fig 6.6):** the working table holds 427,955 (account, depth) rows because the same accounts reappear at every depth and are expanded again each time. The plan shows 187,957 index searches on `transfers_sender_time_idx`, 14,568,858 account lookups for the person filter, and 59.8 million buffer hits; 145 s under EXPLAIN ANALYZE.

Prediction from the plan: searching three levels from each end (bidirectional) avoids the largest frontier levels, so work should fall by orders of magnitude. **Version 2:** forward search to depth ceil(bound/2),

backward search to floor(bound/2), answer = MIN(forward depth + backward depth) over common accounts, person restriction in both searches.

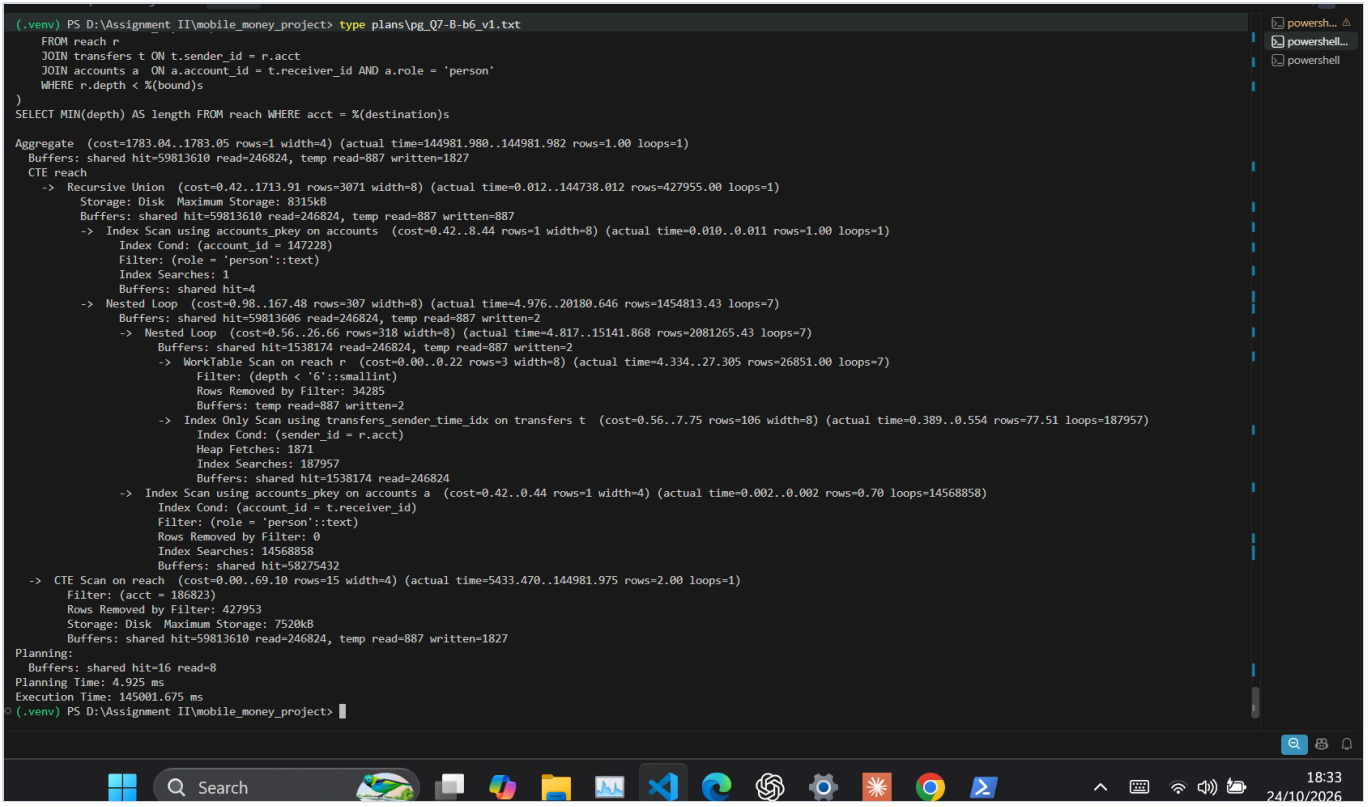


Fig 6.6 — Version 1 plan, Q7-B bound 6: repeated expansion, 14.6 million account lookups, 145,002 ms.

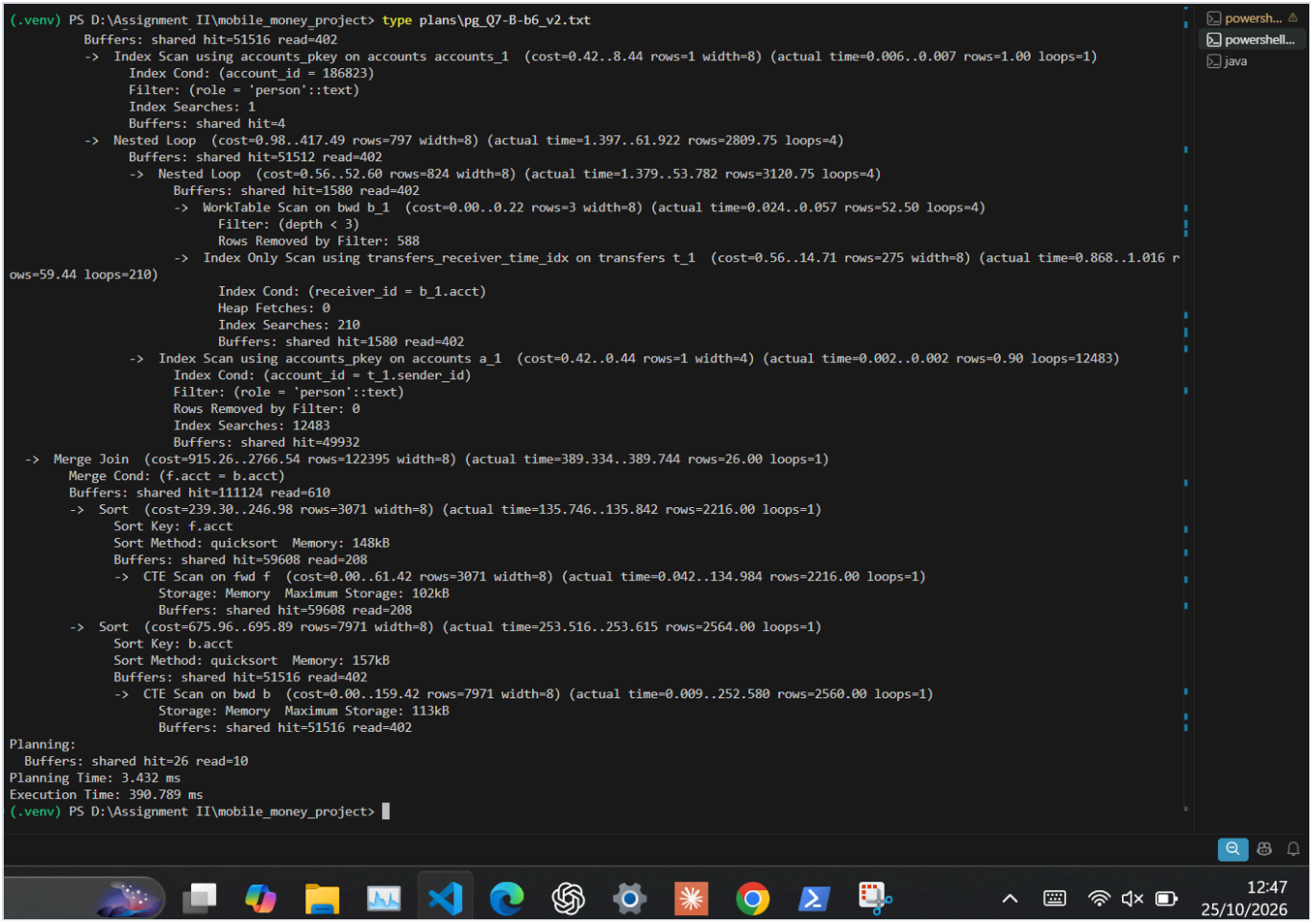


Fig 6.7 — Version 2 plan (second half): forward CTE 2,216 rows, backward CTE 2,560 rows, Merge Join, 111,124 shared buffer hits, 390.8 ms under ANALYZE.

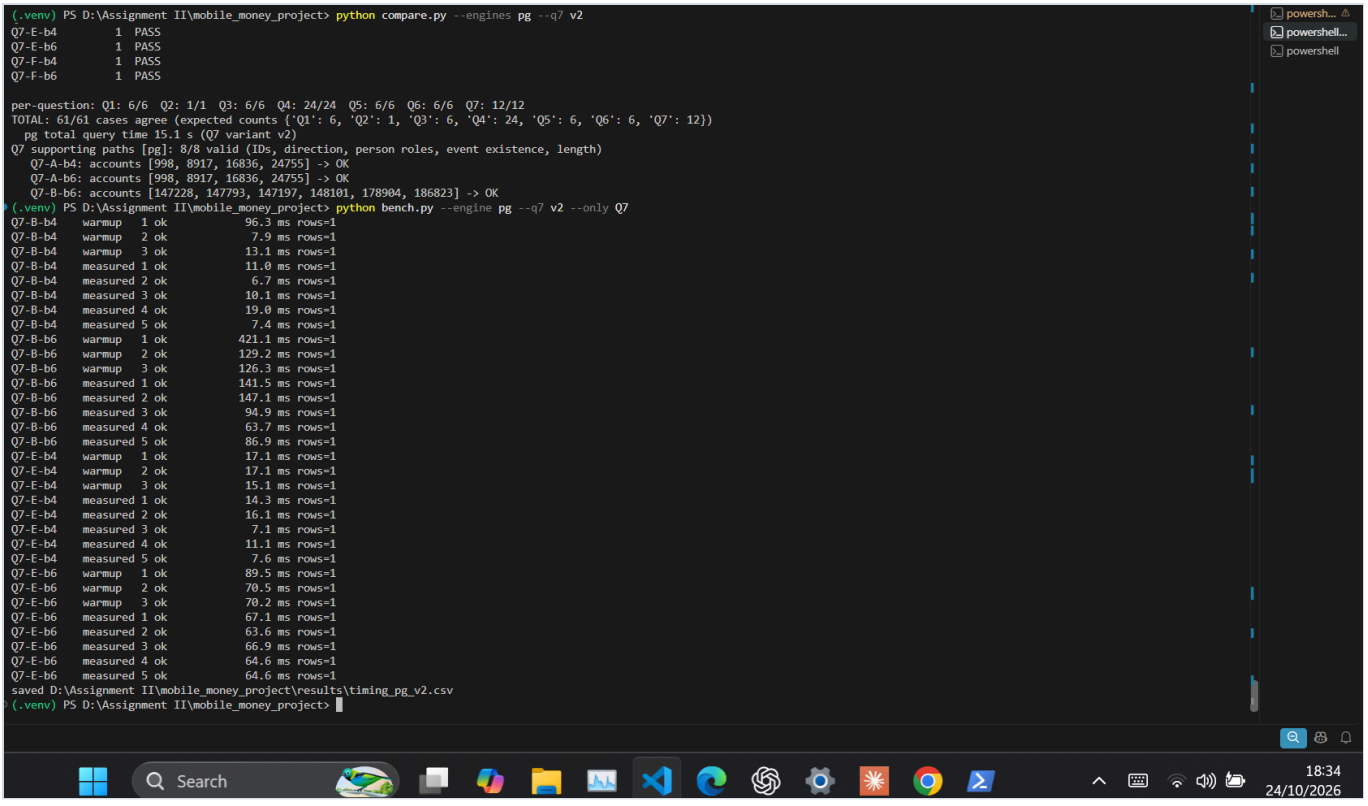


Fig 6.8 — Recheck with version 2: 61/61, Q7 12/12 and supporting paths valid; then `bench.py --engine pg --q7 v2 --only Q7` (four revised cases, all ok).

Case	v1 median ms	v2 median ms	Neo4j median ms	v1 / v2
Q7-B-b4	380.0	10.1	34.0	37.7×
Q7-B-b6	40,731 (1 run)	94.9	54.9	429×
Q7-E-b4	450.3	11.1	35.6	40.4×
Q7-E-b6	timeout (5/5)	64.6	66.1	>929×

Outcome. The prediction held. Work fell from 59.8 million to about 111,000 buffer hits (538× fewer) and every case now completes; version 2 is selected for Section 7 and the final comparison (B/E bound-6 medians 94.9 and 64.6 ms against 40.7 s and a timeout). The plan-time of 390.8 ms exceeds the 94.9 ms median because EXPLAIN ANALYZE adds timing overhead and its single run was less warm (610 pages read). After the rewrite PostgreSQL beats Neo4j for bound 4 (0.30× and 0.31×), is 1.7× slower for B at bound 6 and level for E.

7 Concurrent requests

7.1 Load test

`loadtest.py` uses multiprocessing (spawn), one persistent connection per process and a Barrier. After one warm-up of A-B-C each client cycles A-B-C until a common deadline 15 s after barrier release, finishing any started cycle; 60 s timeout; three repetitions per engine/workload/client count; answers are checked after timing. The Q7 SQL is version 2. p95 = sorted successful latency at position ceil(0.95 n).

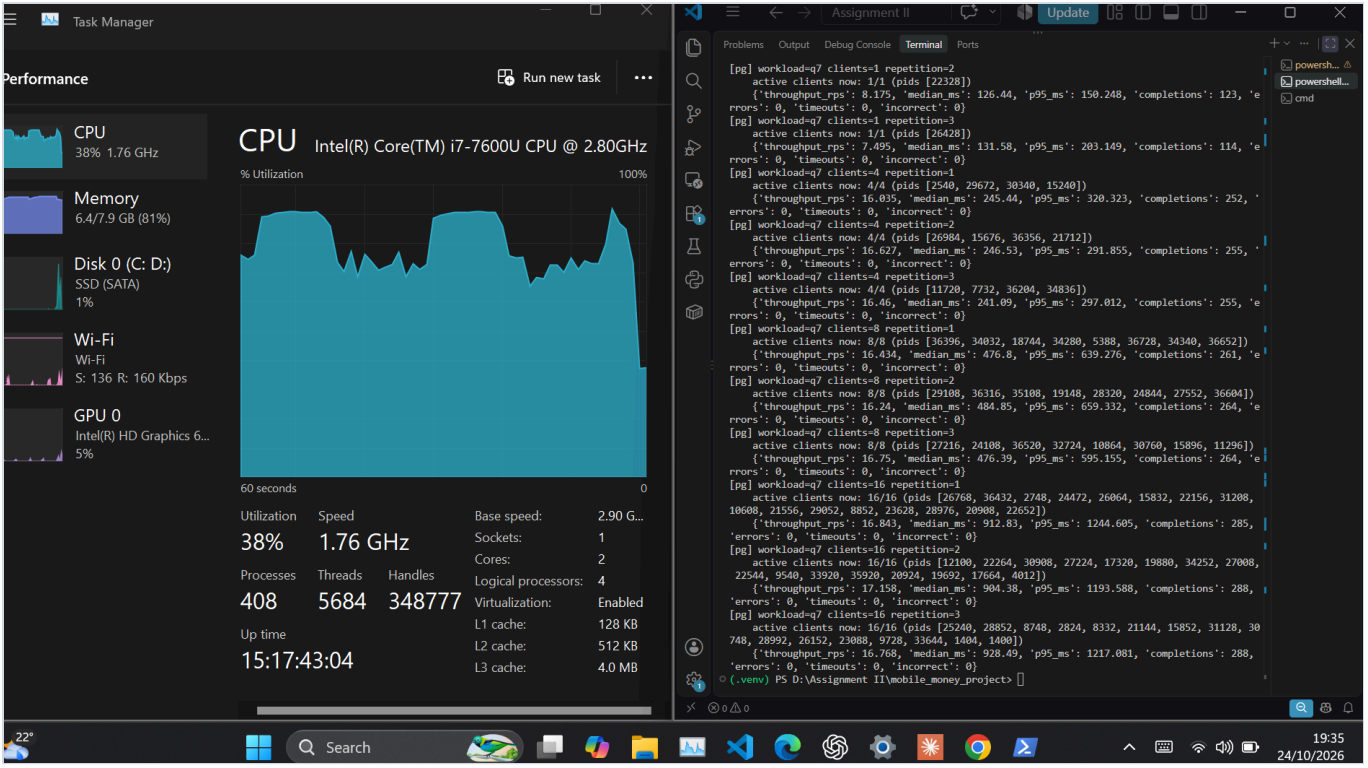


Fig 7.1 — PostgreSQL run with Task Manager: `active clients now: 16/16` and CPU near full utilisation; all repetitions 0 errors, 0 timeouts, 0 incorrect.

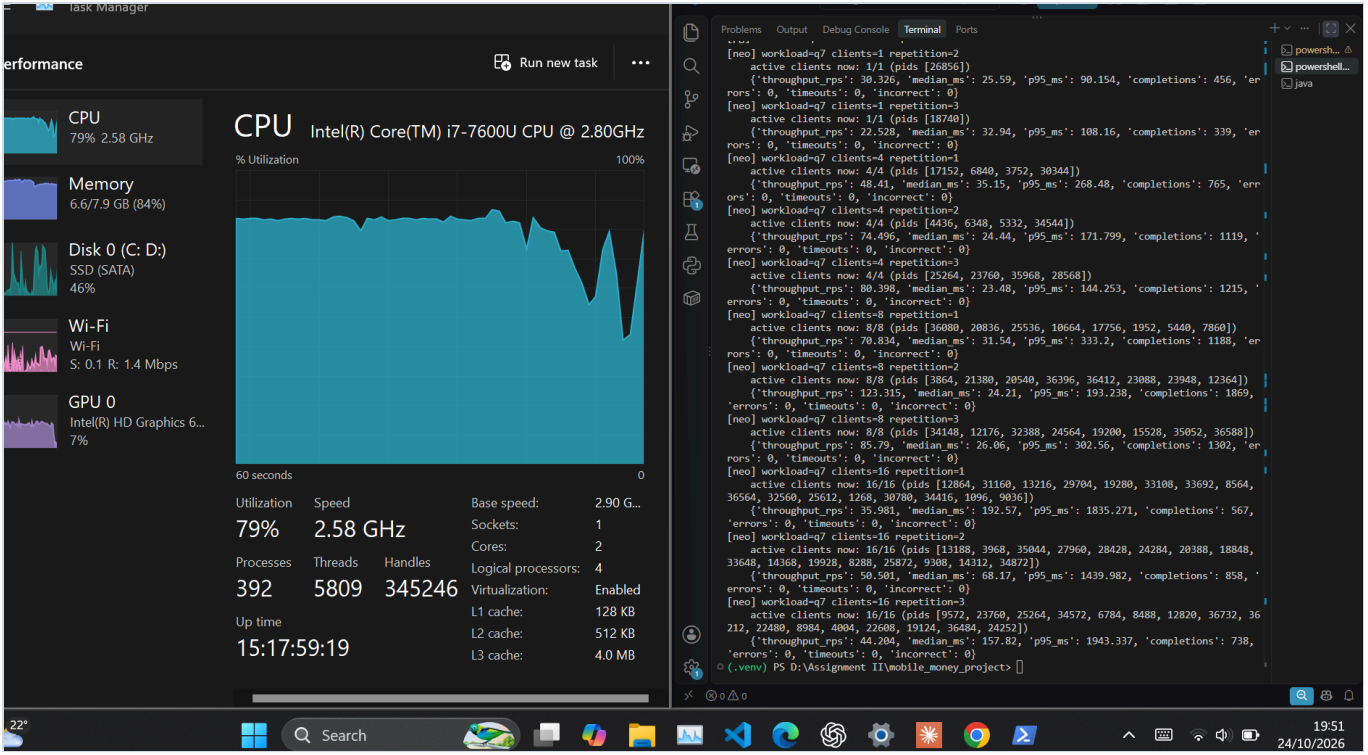


Fig 7.2 — Neo4j run with Task Manager: 16/16 active clients; memory 6.6 of 7.9 GB (84%).

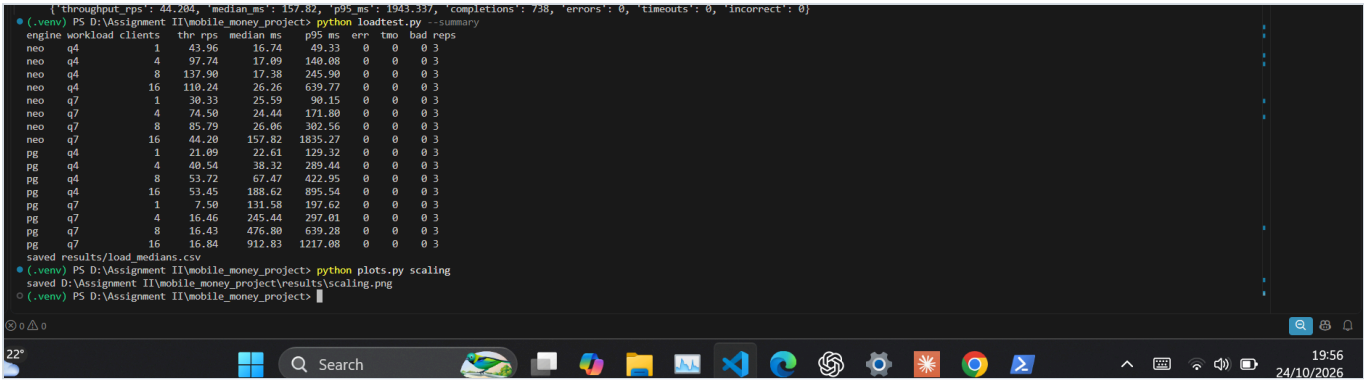


Fig 7.3 — `loadtest.py --summary`: all 16 engine/workload/client combinations (median of three repetitions).

Engine	Workload	Clients	Throughput (req/s)	Median ms	p95 ms	Err	Timeouts	Incorrect
Neo4j	Q4 depth 10	1	44.0	16.7	49	0	0	0
Neo4j	Q4 depth 10	4	97.7	17.1	140	0	0	0
Neo4j	Q4 depth 10	8	137.9	17.4	246	0	0	0
Neo4j	Q4 depth 10	16	110.2	26.3	640	0	0	0
Neo4j	Q7 bound 6	1	30.3	25.6	90	0	0	0
Neo4j	Q7 bound 6	4	74.5	24.4	172	0	0	0
Neo4j	Q7 bound 6	8	85.8	26.1	303	0	0	0
Neo4j	Q7 bound 6	16	44.2	157.8	1835	0	0	0
PostgreSQL	Q4 depth 10	1	21.1	22.6	129	0	0	0
PostgreSQL	Q4 depth 10	4	40.5	38.3	289	0	0	0
PostgreSQL	Q4 depth 10	8	53.7	67.5	423	0	0	0
PostgreSQL	Q4 depth 10	16	53.5	188.6	896	0	0	0
PostgreSQL	Q7 bound 6	1	7.5	131.6	198	0	0	0
PostgreSQL	Q7 bound 6	4	16.5	245.4	297	0	0	0
PostgreSQL	Q7 bound 6	8	16.4	476.8	639	0	0	0
PostgreSQL	Q7 bound 6	16	16.8	912.8	1217	0	0	0

7.2 Scaling

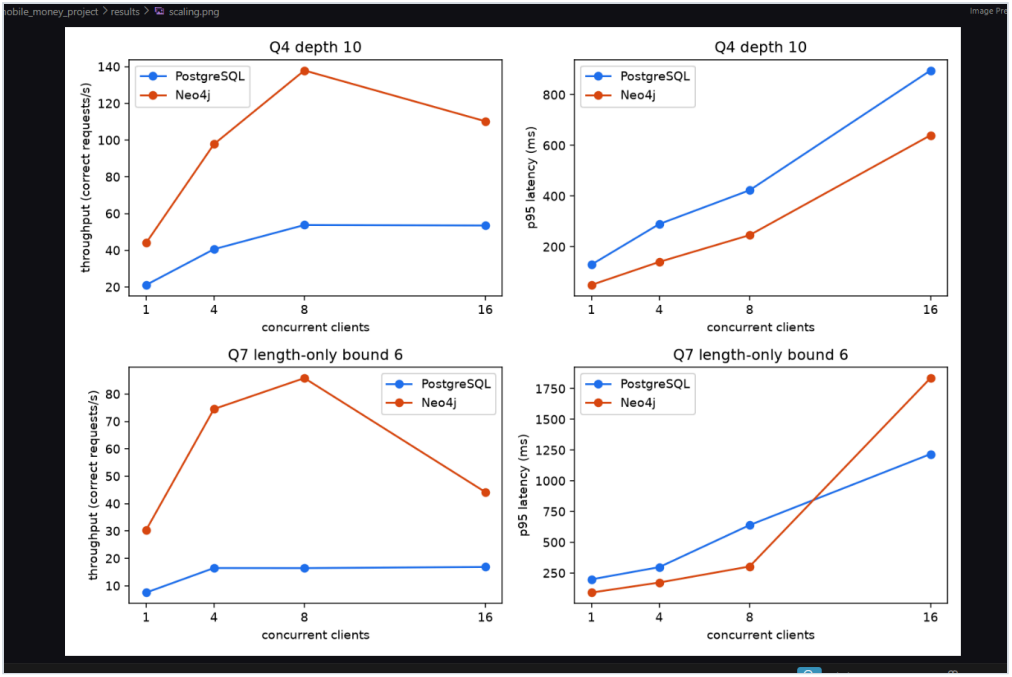


Fig 7.4 — `plots.py scaling`: throughput and p95 against clients (saved in results/scaling.png).

1 versus 16 clients. Q4 depth 10: PostgreSQL 21.1 → 53.5 requests/s (2.5×) with p95 129 → 896 ms; Neo4j 44.0 → 110.2 (2.5×, peak 137.9 at 8) with p95 49 → 640 ms. Q7 bound 6: PostgreSQL 7.5 → 16.8 (2.2×), p95 198 → 1,217 ms; Neo4j 30.3 → 44.2 (peak 85.8 at 8 clients), p95 90 → 1,835 ms.

Saturation. The laptop has four logical processors. PostgreSQL's Q7 throughput is flat at about 16.4–16.8 requests/s from 4 clients, while median latency grows with the client count (245, 477, 913 ms); Little's law (16 / 16.8 = 0.95 s) matches, which is what a saturated CPU with a queue looks like (CPU graph in Fig 7.1). Version 2's plan does 111,000 buffer hits per request, so it is CPU- and memory-bound rather than disk-

bound. Neo4j's Q7 collapse at 16 clients is not explained by its plan; memory was at 84% (Fig 7.2), so memory pressure or garbage collection is a hypothesis we did not test.

Throughput versus response time. Throughput is completed requests per second for the whole group; response time is what one analyst waits. At 16 Q4 clients Neo4j has twice PostgreSQL's throughput and lower p95, but the highest throughput (8 clients) is not the lowest latency (1 client).

Does the graph advantage survive the SQL improvement? In this experiment, partly. The load test already uses version 2 SQL: Neo4j still gives 4.0× the Q7 throughput at 1 client and 5.2× at 8, but its p95 is worse at 16 clients. With an idle system, version 2 removed the Neo4j advantage for E and cut it to 1.7× for B. These findings are limited to one 4-thread laptop, default PostgreSQL settings and three accounts per cycle.

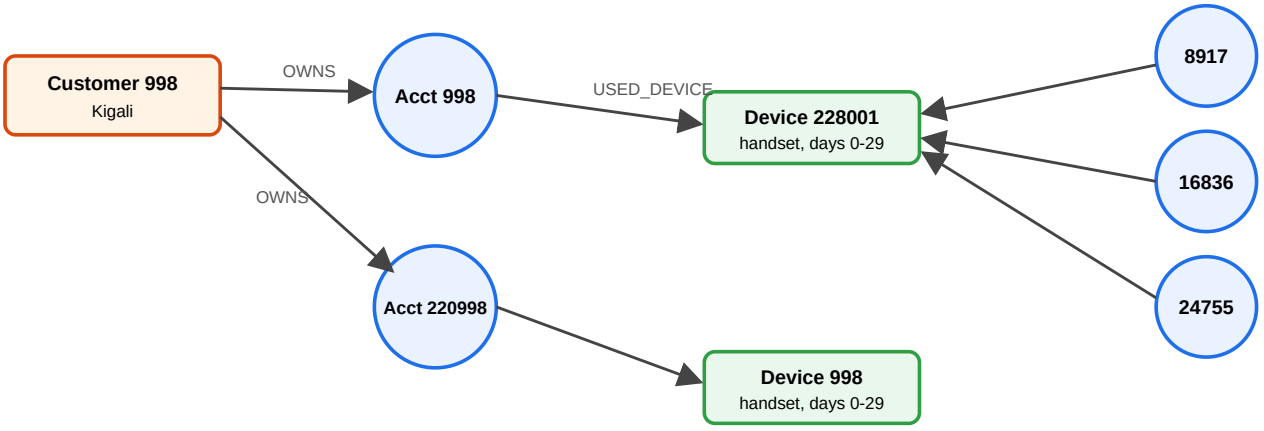
8 Decisions and individual judgement

8.1 Investigation priorities

Pair	Q3 rows	Q4 depth 10	Q5 chains	Q6 cycles	Q7 length (b4 / b6)	Review first
A / D	4 / 4	187 / 192	1 / 0	1 (len 4) / 0	3,3 / 3,3	**A** (998)
B / E	9 / 9	228 / 230	1 / 0	0 / 0	none,5 / none,5	**B** (147228)
C / F	9 / 9	1,433 / 1,426	1 / 0	1 (len 6) / 0	none,5 / none,5	**C** (206978)

Device sharing (Q3), network size (Q4) and shortest-route length (Q7) are nearly identical within each pair, so they do not discriminate. The decisive evidence is Q5/Q6: only A, B and C start a rapid-onward chain, and A and C also return to the start. Q1 amounts point the same way (200,000 vs 45,000; 1,050,000 vs 40,000; 300,000 vs 30,000).

Detail: A (998) versus D (237998), 6 September. A sends 200,000 RWF at 10:00:00 (transfer 20000001) to 8917, which sends 196,000 at 10:00:40, then 16836 sends 192,080 at 10:01:20, and 24755 returns 188,238 RWF to 998 at 10:02:00: four payments in 120 seconds, each 98.0% of the one before. Accounts 998, 8917, 16836 and 24755 are all observed on handset 228001 on days 0–29 (16, 13, 10 and 7 observations). D sends one 45,000 RWF payment at 10:00:00 (transfer 20001651) to 5917; D, 5917, 13836 and 21755 share handset 228301, so D also has a device link, but there is no onward chain or return.



Account 998 and accounts 8917, 16836, 24755 (the whole Q5/Q6 chain) are all observed on handset 228001; 220998 (same owner) uses device 998.

Fig 8.1 — Ownership-device diagram for A from Q3/Q4 results: the full Q5/Q6 chain sits on one shared handset.

The transfer chain/cycle diagram is Fig 5.8; the supporting screenshots are Figs 5.5, 5.7 and 4.3.

Ordinary explanation. Four relatives or members of a savings group (an *ikimina*) may share one handset and pass a collected amount round; 2% per hop could be a fee. **Missing evidence:** the device that initiated each payment (the data say only that the handset was observed), the owners' relationship, payment

8.2 Recommendation

Assignment. Keep PostgreSQL as the system of record and for all reporting (Q1–Q3). Use Neo4j as a read-only graph copy for interactive investigation (Q4–Q7), with PostgreSQL as the cross-check.

Reporting. Q2 (full-month daily totals) took a median 13.1 s in PostgreSQL; all five Neo4j runs hit the 60 s limit. The plans differ: PostgreSQL ran a parallel sequential scan (two workers) over 147,117 pages, while Neo4j scanned 20,003,300 relationships and converted each timestamp row by row (60,009,900 db hits) with no parallel operator in the plan. Simple lookups (Q1) took 1.2 ms versus 14.9–22.1 ms.

Traversal. For Q4 PostgreSQL was faster at every depth, but the median ratio for B rose from 0.085 (depth 4) to 0.625 (depth 10). For Q7 at bound 6 the baseline SQL needed 40.7 s (one completed run for B, none for E) against 54.9 ms and 66.1 ms in Neo4j. That gap came from the SQL: after the bidirectional rewrite PostgreSQL took 94.9 ms (B) and 64.6 ms (E), so Neo4j's lead shrank to 1.7× for B and vanished for E. This is an implementation effect rather than a modelling effect.

Concurrency. With one client Neo4j delivered 2.1× (Q4) and 4.0× (Q7) PostgreSQL's throughput. At 16 clients the Q4 lead persisted (110 versus 53 requests/s), but Neo4j's Q7 throughput fell from 85.8 (8 clients) to 44.2 and its p95 (1,835 ms) exceeded PostgreSQL's (1,217 ms).

Load and storage. PostgreSQL load plus indexing took 280.5 s and 3.73 GB; Neo4j import plus constraints took 259.2 s and 1.61 GB.

Evidence against. Neo4j was never faster for simple reporting or single-user Q4, improved SQL matched it on Q7, it degraded at 16 clients, and two stores cost more to run.

Freshness. Copy new transfers into the graph within 5 minutes. Q5 uses 10-minute gaps, so a longer lag could hide a chain still in progress; stale data gives false negatives and an analyst could close a case on out-of-date evidence.

Limits. (1) One 4-thread, 8 GB laptop, PostgreSQL at default settings (128 MB shared_buffers) against Neo4j with a 3.5 GiB heap, and one synthetic month. (2) Only three accounts per load cycle and two workloads; other queries, larger graphs and Neo4j Enterprise features are untested.

8.3 Individual judgement

Murenzi Prince (101368)

My analysis indicates that the poor performance of the Q7 baseline resulted from the SQL implementation rather than the relational database model itself. For account B at bound 6, only one of five baseline runs completed, taking 40.7 seconds, while the remaining runs timed out. In comparison, Neo4j completed the query in 54.9 ms. The execution plan explains this difference: UNION eliminates duplicate (account, depth) pairs but still allows accounts to be expanded repeatedly at different depths. Consequently, the query performed 187,957 sender-index searches, 14,568,858 account lookups, and approximately 59.8 million buffer hits.

The bidirectional approach reduced this workload by exploring three connections forward and three backward, requiring only around 111,000 buffer accesses. Its median execution time dropped to 94.9 ms. As a result, Neo4j was only 1.7 times faster for account B and performed similarly for account E (64.6 ms versus 66.1 ms).

One factor that could challenge this conclusion is PostgreSQL's default configuration of 128 MB shared_buffers and 4 MB work_mem. If configuration tuning substantially improves baseline performance, the execution plan may not be the sole explanation. Testing both implementations under optimized settings would help verify this finding.

Mashami Pacifique (101406)

My results show that PostgreSQL outperformed Neo4j at every tested Q4 depth, although its relative advantage decreased as traversal depth increased. For account B, the PostgreSQL-to-Neo4j median execution-time ratio increased from 0.085 at depth 4 to 0.625 at depth 10. Account E showed a similar pattern, rising from 0.170 to 0.529. PostgreSQL execution time increased from 4.2 ms to 21.5 ms as the number of reachable accounts grew from 46 to 228, whereas Neo4j's execution times ranged between 34 and 123 ms.

This difference may result from Neo4j's relatively fixed query overhead, also evident in Q1, where execution took 14.9–22.1 ms compared with PostgreSQL's 1.2 ms. Neo4j's additional expansion cost was relatively small, involving 453 nodes and 1,885 database hits, while PostgreSQL incurred increasing costs from recursive index lookups.

However, Neo4j's 123 ms median at depth 6 exceeded its 58 ms result at depth 8, suggesting considerable variability between runs.

To validate the trend, I would repeat depth-10 tests using larger account sets C and F, containing 1,433 and 1,426 accounts respectively, and conduct more than five runs. Consistently superior PostgreSQL performance would suggest that the earlier trend resulted from small datasets.

Delice Sebaganwa Shami (101371)

My findings indicate that Neo4j's throughput decreased when concurrency increased from 8 to 16 clients, whereas PostgreSQL's throughput remained stable. Neo4j's throughput dropped from 85.8 requests per second to 44.2, while its 95th-percentile latency increased from 303 ms to 1,835 ms, exceeding PostgreSQL's 1,217 ms.

PostgreSQL reached a throughput plateau of approximately 16.4 requests per second with four clients, likely reflecting the laptop's four logical processors. Adding more clients increased waiting time, causing median latency to rise from 245 ms to 477 ms and then 913 ms. This behaviour aligns with Little's law: 16 concurrent clients divided by approximately 16.8 requests per second gives an average response time of around 0.95 seconds.

Throughput represents completed requests per second, whereas latency measures the time a user waits for a response. Although Neo4j's declining throughput cannot be directly attributed to its execution plan, Task Manager showed memory usage reaching 84% (6.6 of 7.9 GB). Memory pressure and garbage collection are possible explanations, but neither is confirmed.

To investigate further, I would repeat the 16-client test while recording garbage-collection activity or reducing the Java heap size. If significant pauses do not occur, CPU contention would become a more plausible explanation.

Contributions

Member	Contribution
Murenzi Prince	Installed PostgreSQL 18.6 and Neo4j Community 5.26.32 (Java 17), set the time zone to Africa/Kigali, and recorded the environment (§1.1). Ran `capture.py` (20,993,500 records in 322 s) and the PostgreSQL load (109.69 s load, 170.83 s indexes and constraints) and took the §1 and §2 screenshots. Ran the Q7 baseline versus bidirectional SQL comparison and wrote the §8.3 note on it (40.7 s baseline versus 94.9 ms).
Mashami Pacifique	Ran `prepare_import.py` and the Neo4j full import (704,600 nodes, 20,536,900 relationships), created the constraints and the range index, and ran `verify_neo4j.py` (§3). Compared account 998 in SQL and Cypher (§3.3). Ran Q1 to Q7 in both engines and the 61-case comparison (61/61); checked the A chain (transfers 20000001 to 20000003) and the 4-transfer cycle against the CSV rows (§4, §5, §6.1). Wrote the §8.3 note on the Q4 PostgreSQL/Neo4j ratio.
Delice Sebaganwa Shami	Ran the 21-case timing benchmark one database at a time (3 warm-ups, 5 measured runs) and collected the EXPLAIN and PROFILE plans for Q2, Q4 and Q7 (§6.2). Ran the load tests at 1, 4, 8 and 16 clients for both engines (§7), took the Task Manager screenshots, and wrote the §8.3 note on Neo4j at 16 clients. Pushed the project to GitHub, built the ZIP, and did the final 61-case rerun and the stopped-services screenshots (§9).
LLM	Supported project code, SQL/Cypher queries, the Q7 improvement, setup guidance, report layout, diagrams and draft explanations. The LLM output was run and checked by the group as described below.

LLM / external assistance and how it was checked

An LLM was used throughout to assist with project code (capture, load, import, comparison, benchmark, load-test and plotting programs), SQL and Cypher query pairs including the Q7 bidirectional version, setup instructions, report layout, diagrams and draft explanations for the analysis sections. External sources included the PostgreSQL, Neo4j and Python documentation and the handout feature guides.

Checks. Every member ran commands on their own laptop and took screenshots of their own work; no member relied on a single group laptop. The generator SHA-256 was verified unchanged (Fig 9.1); all 61 cases were compared across PostgreSQL, Neo4j and an independent CSV-based reference (61/61); a chain, a cycle and a device overlap were re-derived from CSV rows; row counts and audit totals were verified in both databases; all numbers in this report are taken from the saved result files and screenshots. Each member has read the report and can explain it.

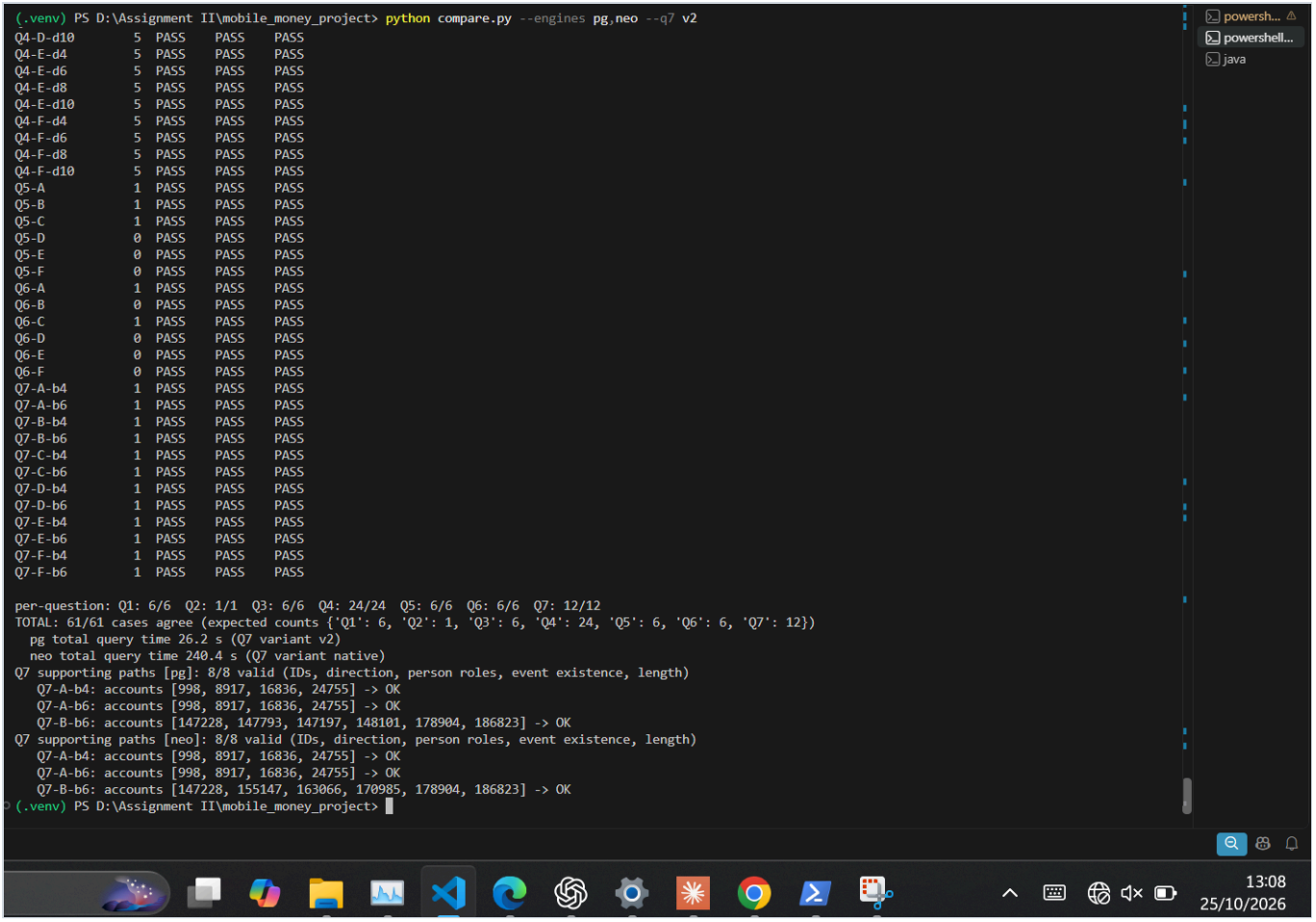


Fig 9.2 — final rerun `python compare.py --engines pg,neo --q7 v2`: TOTAL 61/61 cases agree; Q7 8/8 supporting paths valid in each engine; PostgreSQL 26.2 s, Neo4j 240.4 s.

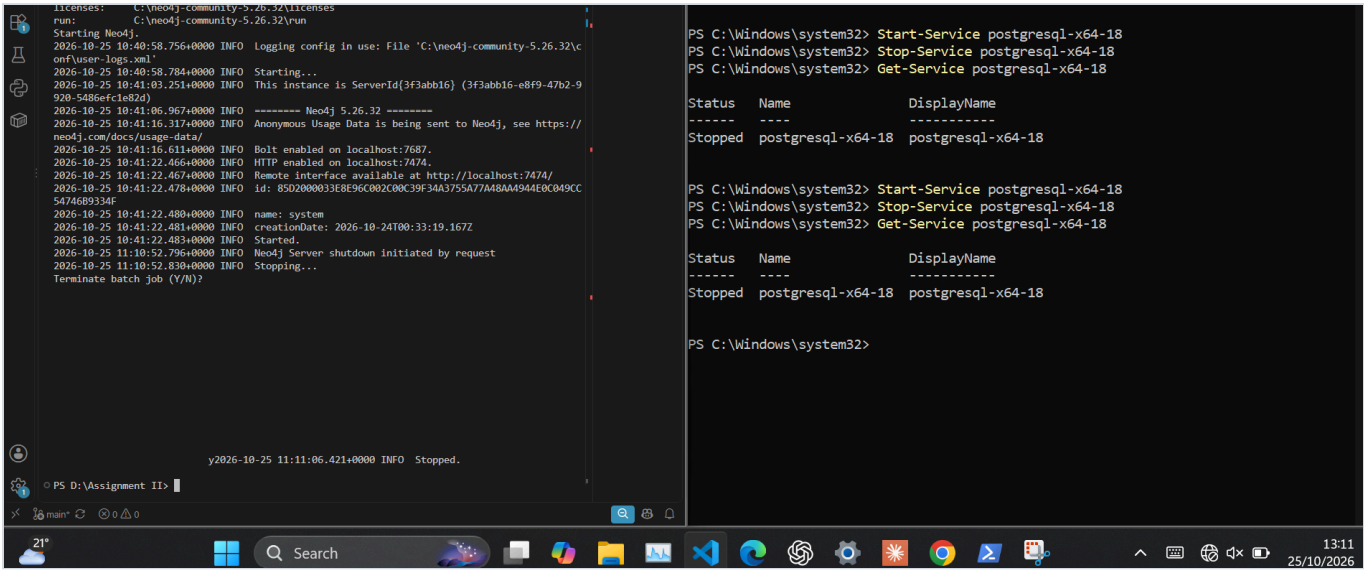


Fig 9.3 — both services stopped: Neo4j shut down ("Stopped.", left) and `Get-Service postgresql-x64-18` reports Stopped (right). The database stores are retained.